DNA十六元基索引系统_V5.0_源码文档 开始完善补正内容 V0.003

对比之前我的个人著作权产品源码，这个著作权工程文件中出现新的函数片段主要有 TVM元基索引的笛卡尔关系分析模块，分词前歧义中文混合数字识别模块和时函数应用模块，因为只有60页空间，于是开始描述下重点。

--目录

****

--源码

1 TVM元基索引的笛卡尔关系分析模块
--主要出现在tinshell的TVM命令句扩展识别。
TVM命令句构造
--LimitedRowAttributesOfColumnsInMemoryClass 举例

```
package OSI.OSU.addAction;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import ME.VPC.M.app.App;
```

```
import OSI.OSU.crab.CrabInterface;
import S_A.pheromone.IMV_SQI;
import jnisort.LYGSortESU9D;
//稍后封装成一个统一的傻瓜接口。
public class LimitedRowAttributesOfColumnsInMemoryClass implements
        CrabInterface {
        String callFunctionKey;
        String className = "LimitedRowAttributesOfColumnsInMemoryClass";
        // public IMV_SQI chromosomeRoot= new IMV_SQI();
        // public IMV_SQI chromosomeFlower= new IMV_SQI();
        // public IMV_SQI chromosomeLeaf= new IMV_SQI();
        // public IMV_SQI chromosomeBlooming= new IMV_SQI();
        // public IMV_SQI chromosomeMetabolism= new IMV_SQI();
        // public IMV_SQI chromosomePDE= new IMV_SQI();
        // public IMV_SQI chromosomeDNA= new IMV_SQI();
        // public IMV_SQI chromosomeNode= new IMV_SQI();
        // 用于表达元基花的链接// 确定元基花的染色体位置// 确定元基花的染色体调用细节
        // 确定染色体的粘合机制// 确定染色体的剥离机制// 确定染色体的静态执行
        // StaticRootMap.chromosomeRoot.put("crab", null);
        // StaticRootMap.chromosomeLeaf.put("crab", null);
        // StaticRootMap.chromosomeDNA.put("crab", null);
        @SuppressWarnings("unchecked")
        public void chromosomes(App NE) {
                NE.app_S.staticRootMap.initMap(NE);
                callFunctionKey = "callFunctionKey";
                // 20230207 走统计新陈代谢
                NE.app_S.staticRootMap.staticBloomingTimes.put(callFunctionKey, (long) 0);
                NE.app_S.staticRootMap.staticBloomingTime.put(callFunctionKey,
                        System.currentTimeMillis());// 增加记忆时间。20241013
                NE.app_S.staticRootMap.staticClass_XE_Map.put(callFunctionKey,"S_AOPM");
                NE.app_S.staticRootMap.chromosomeNode.put(callFunctionKey,
                        new LimitedRowAttributesOfColumnsInMemoryClass());// 20241001准备把这行移出去。
                NE.app_S.staticFunctionMapS_AOPM_C.annotationMap.put(
                        callFunctionKey, "inputValues:传参因子:因子");
                // String callFunctionKey= "callFunctionKey";
                // StaticRootMap.initMap();
        };
        /*
         * 用于表达花语的链接
        // 确定花语的入参模式 // 确定花语的绽放次数 // 确定花语的最优选择 // 确定花语的映射记忆
        */
        // StaticRootMap.chromosomeFlower.put("crab", null);
        // StaticRootMap.chromosomeBlooming.put("crab", null);
        // StaticRootMap.chromosomeMetabolism.put("crab", null);
        @SuppressWarnings("unchecked")
        public void bloomings(App NE) {
                NE.app_S.staticRootMap.chromosomeBlooming.put(callFunctionKey,
                        this.getClass());
        }
```

```
 // 用于表达执行方式和函数内容
// 确定函数的dna编码方式和名称// 确定输入的计算参数名称// 确定输出的结果对象类型
// 确定函数的三方资源// 确定函数的加密形式// 确定函数的运算周期
// StaticRootMap.chromosomeNode.put("crab", null);
// StaticRootMap.chromosomePDE.put("crab", null);
public void neroCells() {
};
/*
 * 用于表达执行主体
 */
//
// StaticRootMap.chromosomeBlooming.put("crab", null);
// StaticRootMap.chromosomeRNA.put("crab", null);
// return null;
@SuppressWarnings({ "rawtypes", "unchecked", "unused" })
public boolean logic(IMV_SQI inputValues, String[] 传参因子, int 因子,
        App NE, IMV_SQI outputReg) {
        S_logger.Log.logger.info("" + "400-size-02-"+ NE.app_S.workVerbalMap.command_V.cartesianLooped
                .size());
        if (NE.app_S.workVerbalMap.command_V.cartesianLooped.contains(className)) {
                S_logger.Log.logger.info("" + "400-size-01-"
                + NE.app_S.workVerbalMap.command_V.countReject++);
                return false;
        }
        if (!NE.app_S.workVerbalMap.command_V.command.contains("行")) {
                return false;
        }
        S_logger.Log.logger.info("" + "LimitedRow-string-400-00-->\n");
        Iterator<String> iterators = NE.app_S.workVerbalMap.command_V.cartesianWorkActionsRightsSV
                .keySet().iterator();
        String fromValue = "";
        String toValue = "";
        // 逻辑分解增加精度
        boolean needFind = false;
        while (iterators.hasNext()) {
                String string = iterators.next();
                S_logger.Log.logger.info("" + "LimitedRow-string-400-01-->"+ string);
                if (string.contains("V+行")) {
                        needFind = true;
                        break;
                }
        }
        List<String> fromValues = new ArrayList<>();
        List<String> toValues = new ArrayList<>();
        if (needFind) {
                iterators = NE.app_S.workVerbalMap.command_V.cartesianWorkActionsRightsVO
                        .keySet().iterator();
                while (iterators.hasNext()) {
                        String string = iterators.next();
```

```
                if (string.contains("从-")) {
                        // 1
                        String[] strings = string.split("-");
                        // 2
                        if (strings.length > 1) {
                                boolean isNumeric = true;
                                for (int i = 0; i < strings[1].length(); i++) {
                                        if (strings[1].charAt(i) < 48|| strings[1].charAt(i) > 57) {
                                                isNumeric = false;
                                        }
                                }// 3
                                if (isNumeric) {
                                        fromValue = strings[1];
                                        fromValues.add(string);
                                        /*
                                         * 相同数多的情况下，需要将fromValue +精度变
                                         * 成一个list，选择最短的条件进行输出。
                                         */
                                }
                        }// 4
                }
                if (string.contains("到-")) {
                        // 1
                        String[] strings = string.split("-");
                        // 2
                        if (strings.length > 1) {
                                boolean isNumeric = true;
                                for (int i = 0; i < strings[1].length(); i++) {
                                        if (strings[1].charAt(i) < 48|| strings[1].charAt(i) > 57) {
                                                isNumeric = false;
                                        }
                                }// 3
                                if (isNumeric) {
                                        toValue = strings[1];
                                        /*
                                         * 同理 相同数多的情况下，需要将fromValue +精度变
                                         * 成一个list，选择最短的条件进行输出。
                                         * 关于源码的循序渐进设计思维方式，价值思考 --later
                                         */
                                        toValues.add(string);
                                }
                        }
                }
        }
}
// 排序逻辑
if (fromValues.isEmpty() || toValues.isEmpty()) {
        return false;
}
```

```
		//下面这种排序思维不适用于单行命令句中的数字识别，因为不会再出现有复杂笛卡尔条件。
		String[] fromValueStrings = new String[fromValues.size()];
		int[] fromValueStringRights = new int[fromValues.size()];
		String[] toValueStrings = new String[toValues.size()];
		int[] toValueStringRights = new int[toValues.size()];
		/*
		 * size大就用iterator，自己去探索为什么。
		 **/
		for (int i = 0; i < fromValues.size(); i++) {
			fromValueStrings[i] = fromValues.get(i);
			String temp = NE.app_S.workVerbalMap.command_V.cartesianWorkActionsRightsVO
					.getString(fromValueStrings[i]);
			fromValueStringRights[i] = Integer.valueOf(temp);
		}
		for (int i = 0; i < toValues.size(); i++) {
			toValueStrings[i] = toValues.get(i);
			String temp = NE.app_S.workVerbalMap.command_V.cartesianWorkActionsRightsVO
					.getString(toValueStrings[i]);
			toValueStringRights[i] = Integer.valueOf(temp);
		}
		// sort
		new LYGSortESU9D().javaSort(fromValueStringRights,fromValueStrings);
		new LYGSortESU9D().javaSort(toValueStringRights,toValueStrings);
		// 3 信息组合 成指令集术语
		String shellType = "操作:" + fromValueStrings[0].split("-")[1]
				+ "|行至|" + toValueStrings[0].split("-")[1] + "";
		// 4 输出
		//String shellType = "操作:" + fromValues.get(0)
		//+ "|行至|" + toValues.get(0) + "";
		String[] strings = shellType.split(":");
		List<String[]> list = new ArrayList();
		list.add(strings);
		NE._I_U.outputMap.put("操作", list);// 集成到老的接口模式先，避免bug*/
		NE._I_U.outputMap.put("type", "进行选择");
		//register
		NE._I_U.sets = strings[1].split("\\|");
		//当年我犯了个错误，合并了多个工程调通后的程序没有及时的注释掉，导致莫名乱码UTF8
		//乱码问题
		//后import出错选择了没有注释掉的源码。
		//我当时不注释的原因是将来用得到加上通过入参不同来确定函数唯一。
		NE.app_S.workVerbalMap.command_V.cartesianLooped.put(
				className, "");
		return true;
	}
}
```

TVM命令句扩展

```
package S_A.SixActionMap;
import ME.VPC.M.app.App;
import OSI.OSU.addAction.AddActionInterfaceClass;
```

```
import OSI.OSU.addAction.AddFindColumnsInMemoryClass;
import OSI.OSU.addAction.AddParserMixedStringClass;
import OSI.OSU.addAction.AddParserMixedStringToListClass;
import OSI.OSU.addAction.LimitedRowAttributesOfColumnsInMemoryClass;
import OSI.OSU.addAction.UpdateColorAttributesOfColumnsInMemoryClass;
import S_A.pheromone.IMV_SQI;
@SuppressWarnings("unchecked")
public class CreativeVerbalMap {
	public IMV_SQI flowerActionMap = new IMV_SQI();
	public static void initInitonActions(App NE) {
		regAddActionInterfaceClass(NE);
		regAddParserMixedStringClass(NE);
		regAddParserMixedStringToListClass(NE);
		regAddFindColumnsInMemoryClass(NE);
		regUpdateColorAttributesOfColumnsInMemoryClass(NE);
		regLimitedRowAttributesOfColumnsInMemoryClass(NE);
		/*
		 * 函数注册越来越多，以后可以OSGI插件化，也可以函数分层。
		 */
	}
	static void regAddActionInterfaceClass(App NE) {
		// TODO Auto-generated method stub
		// 非OSGI模式注册花语言，其他见我著作权的CrabInterfaceClass。
		NE.app_S.flowerAction.FlowerSixDomainActions.put("I+表",
				"findTableInMemory");
		// 注册花函数
		AddActionInterfaceClass addActionInterfaceClass
		= new AddActionInterfaceClass();
		String callFunctionKey = "findTableInMemory";
		// 20230207 走统计新陈代谢, 见CrabInterfaceClass
		// chromosomes
		NE.app_S.staticRootMap.staticBloomingTimes.put(
				callFunctionKey, (long) 0);
		NE.app_S.staticRootMap.staticBloomingTime.put(callFunctionKey,
				System.currentTimeMillis());// 增加记忆时间。20241013
		NE.app_S.staticRootMap.staticClass_XE_Map.put(callFunctionKey, "S_AOPM");
		NE.app_S.staticRootMap.chromosomeNode.put(callFunctionKey,
				addActionInterfaceClass);// 20241001准备把这行移出去。
		NE.app_S.staticFunctionMapS_AOPM_C.annotationMap.put(
				callFunctionKey, "inputValues:传参因子:因子");
		// booming
		NE.app_S.staticRootMap.chromosomeBlooming.put(callFunctionKey,
				addActionInterfaceClass);
	}
	static void regAddParserMixedStringClass(App NE) {
		// 注册花函数
		AddParserMixedStringClass addParserMixedStringClass
		= new AddParserMixedStringClass();
		String callFunctionKeyaddParserMixedStringClass
```

```
            = "parserMixedString";
            //
            NE.app_S.flowerAction.FlowerChromosomeActions.put(
                    callFunctionKeyaddParserMixedStringClass,
                    addParserMixedStringClass);
            // 20230207 走统计新陈代谢, 见CrabInterfaceClass
            // chromosomes
            NE.app_S.staticRootMap.staticBloomingTimes.put(callFunctionKeyaddParserMixedStringClass, (long) 0);
            NE.app_S.staticRootMap.staticBloomingTime.put(callFunctionKeyaddParserMixedStringClass, System
                            .currentTimeMillis());// 增加记忆时间。20241013
            NE.app_S.staticRootMap.staticClass_XE_Map.put(
                    callFunctionKeyaddParserMixedStringClass, "A_VECS");
            NE.app_S.staticRootMap.chromosomeNode.put(callFunctionKeyaddParserMixedStringClass,
                    addParserMixedStringClass);// 20241001准备把这行移出去。
            NE.app_S.staticFunctionMapS_AOPM_C.annotationMap.put(callFunctionKeyaddParserMixedStringClass,
                    "inputValues:传参因子:因子");
            // booming
            NE.app_S.staticRootMap.chromosomeBlooming.put(callFunctionKeyaddParserMixedStringClass,
                    addParserMixedStringClass);
    }
    static void regAddParserMixedStringToListClass(App NE) {
            // 注册花函数
            AddParserMixedStringToListClass addParserMixedStringToListClass
            = new AddParserMixedStringToListClass();
            String callFunctionKeyAddParserMixedStringToListClass
            = "parserMixedStringToList";
            //
            NE.app_S.flowerAction.FlowerChromosomeActions.put(callFunctionKeyAddParserMixedStringToListClass,
                    addParserMixedStringToListClass);
            // 20230207 走统计新陈代谢, 见CrabInterfaceClass
            // chromosomes
            NE.app_S.staticRootMap.staticBloomingTimes.put(
                    callFunctionKeyAddParserMixedStringToListClass, (long) 0);
            NE.app_S.staticRootMap.staticBloomingTime.put(callFunctionKeyAddParserMixedStringToListClass
                    , System.currentTimeMillis());// 增加记忆时间。20241013
            NE.app_S.staticRootMap.staticClass_XE_Map.put(
                    callFunctionKeyAddParserMixedStringToListClass, "A_VECS");
            NE.app_S.staticRootMap.chromosomeNode.put(
                    callFunctionKeyAddParserMixedStringToListClass,
                    addParserMixedStringToListClass);// 20241001准备把这行移出去。
            NE.app_S.staticFunctionMapS_AOPM_C.annotationMap.put(
                    callFunctionKeyAddParserMixedStringToListClass,
                    "inputValues:传参因子:因子");
            // booming
            NE.app_S.staticRootMap.chromosomeBlooming.put(
                    callFunctionKeyAddParserMixedStringToListClass,
                    addParserMixedStringToListClass);
    }
    /*
```

```
	 * important key = '输出-内容' '仅含-' '+列', those three keys could increase
	 * to a combination key ,and '+列' is a fit rights key, and '输出-内容' '仅含-'
	 * are complement keys.  --let's ..
	 * */
	static void regAddFindColumnsInMemoryClass(App NE) {
		// VI+列 later
		NE.app_S.flowerAction.FlowerSixDomainActions.put("+列",
			"addFindColumnsInMemoryClass");
		// 注册花函数
		AddFindColumnsInMemoryClass addFindColumnsInMemoryClass
		= new AddFindColumnsInMemoryClass();
		String callFunctionKeyAddFindColumnsInMemoryClass
		= "addFindColumnsInMemoryClass";
		// 20230207 走统计新陈代谢, 见CrabInterfaceClass
		// chromosomes
		NE.app_S.staticRootMap.staticBloomingTimes.put(
			callFunctionKeyAddFindColumnsInMemoryClass, (long) 0);
		NE.app_S.staticRootMap.staticBloomingTime.put(
			callFunctionKeyAddFindColumnsInMemoryClass, System
				.currentTimeMillis());// 增加记忆时间。20241013
		NE.app_S.staticRootMap.staticClass_XE_Map.put(
			callFunctionKeyAddFindColumnsInMemoryClass, "S_AOPM");
		NE.app_S.staticRootMap.chromosomeNode.put(
			callFunctionKeyAddFindColumnsInMemoryClass,
			addFindColumnsInMemoryClass);// 20241001准备把这行移出去。
		NE.app_S.staticFunctionMapS_AOPM_C.annotationMap.put(
			callFunctionKeyAddFindColumnsInMemoryClass,
			"inputValues:传参因子:因子");
		// booming
		NE.app_S.staticRootMap.chromosomeBlooming.put(
			callFunctionKeyAddFindColumnsInMemoryClass,
			addFindColumnsInMemoryClass);
	}
	static void regUpdateColorAttributesOfColumnsInMemoryClass(
		App NE) {
		NE.app_S.flowerAction.FlowerSixDomainActions.put("E+颜色",
			"updateColorAttributesOfColumnsInMemoryClass");
		// 注册花函数
		UpdateColorAttributesOfColumnsInMemoryClass
		updateColorAttributesOfColumnsInMemoryClass
		= new UpdateColorAttributesOfColumnsInMemoryClass();
		String callFunctionKeyUpdateColorAttributesOfColumnsInMemoryClass
		= "updateColorAttributesOfColumnsInMemoryClass";
		// 20230207 走统计新陈代谢, 见CrabInterfaceClass
		// chromosomes
		/*
		 * 之后走_SMV 这里的变量中入参将全部省略，避免内存占用浪费。之后可以设计个temp区间
		 * 当作类脑存储容器专门用于计算失效的 和 临时的变量，关于这类格式
		 * -- trif binlog，map，DB，json
```

```
		 * 等。我会首先采用map 不累。。
		 */
		NE.app_S.staticRootMap.staticBloomingTimes.put(
				callFunctionKeyUpdateColorAttributesOfColumnsInMemoryClass,
				(long) 0);
		NE.app_S.staticRootMap.staticBloomingTime.put(
				callFunctionKeyUpdateColorAttributesOfColumnsInMemoryClass,
				System.currentTimeMillis());// 增加记忆时间。20241013
		NE.app_S.staticRootMap.staticClass_XE_Map.put(
				callFunctionKeyUpdateColorAttributesOfColumnsInMemoryClass,
				"S_AOPM");
		NE.app_S.staticRootMap.chromosomeNode.put(
				callFunctionKeyUpdateColorAttributesOfColumnsInMemoryClass,
				updateColorAttributesOfColumnsInMemoryClass);// 20241001准备把这行移出去。
		NE.app_S.staticFunctionMapS_AOPM_C.annotationMap.put(
				callFunctionKeyUpdateColorAttributesOfColumnsInMemoryClass,
				"inputValues:传参因子:因子");
		// booming
		NE.app_S.staticRootMap.chromosomeBlooming.put(
				callFunctionKeyUpdateColorAttributesOfColumnsInMemoryClass,
				updateColorAttributesOfColumnsInMemoryClass);
	}
	static void regLimitedRowAttributesOfColumnsInMemoryClass(App NE) {
		/*
		 * 开始构造数字行数提取指令 从- 组合 到- 然后分析 从- 组合 到- 展示+行 仅+展示
		 */
		NE.app_S.flowerAction.FlowerSixDomainActions.put("仅+V",
				"P_AggregationLimitMap");
		// 注册花函数
		LimitedRowAttributesOfColumnsInMemoryClass limitedRowAttributesOfColumnsInMemoryClass
		= new LimitedRowAttributesOfColumnsInMemoryClass();
		String callFunctionKeyLimitedRowAttributesOfColumnsInMemoryClass
		= "P_AggregationLimitMap";
		// 20230207 走统计新陈代谢, 见CrabInterfaceClass
		// chromosomes
		/*
		 * 之后走_SMV 这里的变量中入参将全部省略，避免内存占用浪费。之后可以设计个
		 * temp区间当作类脑存储容器专门用于计算失效的 和 临时的变量，关于这类格式
		 * -- trif binlog，map，DB，json
		 * 等。我会首先采用map 不累。。
		 */
		NE.app_S.staticRootMap.staticBloomingTimes.put(
				callFunctionKeyLimitedRowAttributesOfColumnsInMemoryClass,
				(long) 0);
		NE.app_S.staticRootMap.staticBloomingTime.put(
				callFunctionKeyLimitedRowAttributesOfColumnsInMemoryClass,
				System.currentTimeMillis());
		// 增加记忆时间。20241013
		NE.app_S.staticRootMap.staticClass_XE_Map.put(
```

```
                callFunctionKeyLimitedRowAttributesOfColumnsInMemoryClass,
                "O_VECS");
        NE.app_S.staticRootMap.chromosomeNode.put(
                callFunctionKeyLimitedRowAttributesOfColumnsInMemoryClass,
                limitedRowAttributesOfColumnsInMemoryClass);
        NE.app_S.staticFunctionMapO_VECS_C.annotationMap.put(
                callFunctionKeyLimitedRowAttributesOfColumnsInMemoryClass,
                "inputValues:传参因子:因子");
        // booming
        NE.app_S.staticRootMap.chromosomeBlooming.put(
                callFunctionKeyLimitedRowAttributesOfColumnsInMemoryClass,
                limitedRowAttributesOfColumnsInMemoryClass);
    }
}
**********************************************************************************************************
package O_V.OSM.shell;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import ME.VPC.M.app.App;
import S_A.pheromone.IMV_SQI;
import S_A.pheromone.IMV_SQI_SS;
import S_A.pheromone.IMV_SQI_S_;
public class CommandClass {
    //用了NE， 六元分析机可以略去。
    public App regNE;
    // 工作机
    //public WorkVerbalMap workVerbalMap;
    // 单句的命令
    public String command;
    public int countReject = 0;
    // 单句的延伸
    public String commandAcknowledge;
    // 单句的延伸
    public String commandWithNumFilters;
    // 单句的多种延伸
    public String[] acknowledge;
    public String[] acknowledgeSwap;
    public String[] combianationKey;
    // 单句的变换延伸
    public String commandSwap;
    // 单句的分解延伸
    public List<String> commandLists;
    //
    public String[] stringSets;
    public Map<String, String[]> stringSetsMap = new HashMap<>();
    // 单句的预处理
    /*
```

```
 * 将QI颠倒下位置，避免各类社会人问题。
 * --罗瑶光
 * */
public List<String> _IMV_SQI_SS_ = new ArrayList<>();
public List<String> _IMV_SQI_SS_Q_ = new ArrayList<>();
/*
 * --稍后这些关于当前的命令句对应的workVerbalMap中需要clear的对象都走这个类，
 * 免得以后修改或者增加条件，不谨慎漏了几个clear。我认为这是一种计算关系的优 化方式，
 * 属于聚类优化计算逻辑。
 * --随着条件越来越多，之后会统一组合优化这个map关系，然后剔除掉一些成员变量。
 * --trif later --罗瑶光
 */
public IMV_SQI_SS _IMV_SQI_SS = new IMV_SQI_SS();
public IMV_SQI_SS _IMV_SQI_SS_temp = new IMV_SQI_SS();
public IMV_SQI_SS relationsASCII = new IMV_SQI_SS();
public IMV_SQI_SS relationsGrammar = new IMV_SQI_SS();
public IMV_SQI_SS relationsEnglish = new IMV_SQI_SS();
public IMV_SQI_S_ _IMV_SQI_S_ = new IMV_SQI_S_();
public IMV_SQI unknown_map = new IMV_SQI();
public IMV_SQI cartesianLooped = new IMV_SQI();
/*
 * 关系大幅减少仅仅代表计算关系的优化比较通畅，并不代表计算属性的优化是完美的，
 * 于是开始细化计算关系，确定计算属性的重心价值。于是开始拆解 +- SVO关系。
 * 这里分解后，所有涉及这个逻辑的都要分解。于是跟进优化。 --罗瑶光
 */
public IMV_SQI cartesianWorkActionsRightsParserVO = new IMV_SQI();//
public IMV_SQI cartesianWorkActionsRightsParserSV = new IMV_SQI();//
public IMV_SQI cartesianWorkActionsRightsSV = new IMV_SQI();//
public IMV_SQI cartesianWorkActionsPositionsSV = new IMV_SQI();//
public IMV_SQI cartesianWorkActions_posSV = new IMV_SQI();//
public IMV_SQI cartesianWorkActionsRightsVO = new IMV_SQI();//
public IMV_SQI cartesianWorkActionsPositionsVO = new IMV_SQI();//
public IMV_SQI cartesianWorkActions_posVO = new IMV_SQI();//
public IMV_SQI numericsFromUnknownString = new IMV_SQI();
public IMV_SQI alfabeticsFromUnknownString = new IMV_SQI();
public IMV_SQI symbolicsFromUnknownString = new IMV_SQI();
public IMV_SQI unicodesFromUnknownString = new IMV_SQI();
@SuppressWarnings("unchecked")
public void getNumericsFromUnknownMap(String inputString) {
        // 1 阿拉伯字母句
        String string = "";
        int fixOrder = 0;
        for (int i = 0; i < inputString.length(); i++) {
                if (inputString.charAt(i) < 48 || inputString.charAt(i) > 57) {
                        if (!string.isEmpty()) {
                                S_logger.Log.logger.info("" + string + "--"+ fixOrder);
                                // println函数走图形打印机，并发工程记得注释掉或者用其他的classic观测API
                                numericsFromUnknownString.put(string.toString(),
                                        fixOrder++);
```

```
                                string = "";
                        }
                        continue;
                }
                string += inputString.charAt(i);
        }
        if (!string.isEmpty()) {
                S_logger.Log.logger.info("" + string + "--" + fixOrder);
                // println函数走图形打印机，并发工程记得注释掉或者用其他的classic观测API
                numericsFromUnknownString.put(string.toString(),fixOrder++);
                string = "";
        }
        // 2 其他语义，如中文
        // to do later。。
}
@SuppressWarnings("unchecked")
public void getAlfabeticsFromUnknownMap(String inputString) {
        // 英文字母句
        String string = "";
        int fixOrder = 0;
        for (int i = 0; i < inputString.length(); i++) {
                if ((inputString.charAt(i) > 64 && inputString.charAt(
                        i) < 91) || (inputString.charAt(i) > 96 && inputString
                                .charAt(i) < 123)) {
                        string += inputString.charAt(i);
                } else {
                        if (!string.isEmpty()) {
                                alfabeticsFromUnknownString.put(string.toString(),fixOrder++);
                                string = "";
                        }
                        continue;
                }
        }
        if (!string.isEmpty()) {
                S_logger.Log.logger.info("" + string + "--" + fixOrder);
                alfabeticsFromUnknownString.put(string.toString(), fixOrder++);
                string = "";
        }
}
// 这里的逻辑是构造科学计算器用，华瑞集系统先不cover。
/*
 * 我要做的是识别指令符号，｜;; 以后变量中含有指令符号做string用， 我用HTTP
 * encode去变形，或者元基编码去格式变换，避免被这层逻辑分解即可。 省去一把函数。
 */
public void getSymbolicsFromUnknownMap(String inputString) {
}
public IMV_SQI getUnicodesFromUnknownMap(String inputString) {
        return null;
}
```

```
	public static void main(String[] argv) {
		// 我的系统没有16位大数计算业务，有就按这个逻辑+兆+京条件去解决。
		// 框架都搞好了，直接加函数即可 --罗瑶光
	}
	public String simpleChineseNumberSwap(String chineseNumber) {
		String stringSwap = chineseNumber.replace("壹", "一");
		stringSwap = stringSwap.replace("贰", "二");
		stringSwap = stringSwap.replace("两", "二");
		stringSwap = stringSwap.replace("叁", "三");
		stringSwap = stringSwap.replace("肆", "四");
		stringSwap = stringSwap.replace("伍", "五");
		stringSwap = stringSwap.replace("陆", "六");
		stringSwap = stringSwap.replace("柒", "七");
		stringSwap = stringSwap.replace("捌", "八");
		stringSwap = stringSwap.replace("玖", "九");
		stringSwap = stringSwap.replace("拾", "十");
		stringSwap = stringSwap.replace("佰", "百");
		stringSwap = stringSwap.replace("仟", "千");
		stringSwap = stringSwap.replace("万", "万");
		S_logger.Log.logger.info("" + "简体-->" + stringSwap);
		return stringSwap;
	}
	@SuppressWarnings("unused")
	public String fasterChineseNumberSwap(
		StringBuilder stringBuilder) {
		String chineseNumber = stringBuilder.toString();
		S_logger.Log.logger.info("" + "输入-->" + chineseNumber);
		String stringSwap = simpleChineseNumberSwap(chineseNumber);
		String regWan = "";	String regYi = "";	String reg = "";
		String total1 = "";	String total2 = "";	String total3 = "";
		String total4 = "";	String total5 = "";	String total6 = "";
		String outputYi = "";	String[] stringsYi = stringSwap.split("亿");
		// S_logger.Log.logger.info("" + "stringsYi.length-->" +
		// stringsYi.length);
		if (stringsYi.length > 0) {
			int j = 0;
			for (String stringYi : stringsYi) {
				j++;
				String[] stringsWan = stringYi.split("万");
				// S_logger.Log.logger.info("" + "stringsWan.length-->" +
				// stringsWan.length);
				String outputWan = "";
				if (stringsWan.length > 0) {
					int i = 0;
					for (String stringWan : stringsWan) {
						i++;
						/*
						 * 逻辑分层后，这里只要处理一万以内的组合，大幅减少条件分析。
						 */
```

```
                        // S_logger.Log.logger.info("" + "longWan-->" + stringWan);
                        if (stringWan.isEmpty()) {
                                outputWan += "" + 1;
                                reg = outputWan;
                                total1 = outputWan.toString();
                                continue;
                        }
                        long longWan = processWan(stringWan);
                        // S_logger.Log.logger.info("" + "longWan-->" + longWan);
                        if (outputWan.isEmpty()) {
                                outputWan += "" + longWan;
                                reg = outputWan;
                                total2 = outputWan.toString();
                                continue;
                        }
                        if (longWan < 10) {
                                outputWan += "000";
                        } else if (longWan < 100) {
                                outputWan += "00";
                        } else if (longWan < 1000) {
                                outputWan += "0";
                        }
                        outputWan += longWan;
                        reg = outputWan;
                        total3 = outputWan.toString();
                        S_logger.Log.logger.info("" + "total3-->"
                                + outputWan);
                }
                if (stringYi.contains("万")
                        && stringsWan.length < 2) {
                        outputWan += "0000";
                        reg = outputWan;
                        total4 = outputWan.toString();
                        S_logger.Log.logger.info("" + "total4-->"
                                + outputWan);
                }
        } else {
                // wan to do
                outputWan = "10000";
                reg = outputWan;
                total5 = outputWan.toString();
        }
        if (1 == j) {
                regYi = reg;
        }
        if (2 == j) {
                regWan = reg;
        }
}
```

```
                if (stringSwap.contains("亿") && stringsYi.length < 2) {
                        outputYi += reg + "00000000";
                        total6 = outputYi.toString();
                        S_logger.Log.logger.info("" + "total6-->" + outputYi);
                }
                if (stringSwap.contains("亿") && stringsYi.length > 1) {
                        String temp = regWan;
                        if (regWan.length() < 2) {
                                regYi += "0000000";
                        } else if (regWan.length() < 3) {
                                regYi += "000000";
                        } else if (regWan.length() < 4) {
                                regYi += "00000";
                        } else if (regWan.length() < 5) {
                                regYi += "0000";
                        } else if (regWan.length() < 6) {
                                regYi += "000";
                        } else if (regWan.length() < 7) {
                                regYi += "00";
                        } else if (regWan.length() < 8) {
                                regYi += "0";
                        } else if (regWan.length() < 9) {
                                //regYi += "";
                        } else {
                                temp = "";
                        }
                        regYi += temp;
                        outputYi += regYi;
                        S_logger.Log.logger.info("" + "total7-->" + outputYi);
                }
        } else {
                // yi to do
                outputYi = "100000000";
                S_logger.Log.logger.info("" + "total8-->" + outputYi);

        }
        if (!outputYi.isEmpty()) {return outputYi;}
        if (!total6.isEmpty()) {return total6;}
        if (!total5.isEmpty()) {return total5;}
        if (!total4.isEmpty()) {return total4;}
        if (!total3.isEmpty()) {return total3;}
        if (!total2.isEmpty()) {return total2;}
        if (!total1.isEmpty()) {return total1;}
        return outputYi;
}
private long processWan(String stringWan) {
        // TODO Auto-generated method stub
        long total = 0;
        long totalFinal = 0;
```

```
		long set = 0;
		long value = -1;
		for (int i = 0; i < stringWan.length(); i++) {
			if (stringWan.charAt(i) == '零') {value = 0;}
			if (stringWan.charAt(i) == '一') {value = 1;}
			if (stringWan.charAt(i) == '二') {value = 2;}
			if (stringWan.charAt(i) == '三') {value = 3;}
			if (stringWan.charAt(i) == '四') {value = 4;}
			if (stringWan.charAt(i) == '五') {value = 5;}
			if (stringWan.charAt(i) == '六') {value = 6;}
			if (stringWan.charAt(i) == '七') {value = 7;}
			if (stringWan.charAt(i) == '八') {value = 8;}
			if (stringWan.charAt(i) == '九') {value = 9;}
			if (stringWan.charAt(i) == '十') {
				total = total * 10;
				totalFinal = totalFinal + total;
				total = 0;
				value = -1;
			}
			if (stringWan.charAt(i) == '百') {
				total = total * 100;
				totalFinal = totalFinal + total;
				total = 0;
				value = -1;
			}
			if (stringWan.charAt(i) == '千') {
				total = total * 1000;
				totalFinal = totalFinal + total;
				total = 0;
				value = -1;
			}
			// S_logger.Log.logger.info("" + "loop-->" + total);
			if (-1 != value) {
				total = total * 10 + value;
			}
		}
		if (-1 != value) {
			//total += value;
			totalFinal = totalFinal + total;
		}
		return totalFinal;
	}
		public IMV_SQI_SS _IMV_SQI_SS_Q = new IMV_SQI_SS();
	public String commandWithoutNumerics = "";
	public String chineseSimpleCommandWithoutNumerics = "";
	public boolean initedArabicNumber = false;
	/*
	 * 保证相等长度替换，symbolSwapNumerics只能包含一个位的char符号。
	 **/
```

```
	public String symbolSwapNumerics = "*";
	public void initArabicNumber() {
		regNE.app_S.studyVerbalMap.extractNumberfromString(this);
	}
	//400局部输出正确
	//chineseSimpleCommandWithoutNumerics-->在输出的数据表中仅展示从第*行到第*行的数据
	public void fussionOfArabicNumber() {
		// TODO Auto-generated method stub

	}
	public void initSixActions(App NE) {
		// TODO Auto-generated method stub
		this.regNE = NE;
	}
}
```

****

TVM命令句识别

```
package O_V.OSM.shell;
import ME.VPC.M.app.App;
import O_V.OSA.shell.PL_XA_E;
import S_I.OSI.PEI.PCI.PSI.tinShell.TinMap;
import S_I.OSI.PSO.regex.DoSplit;
import S_A.pheromone.IMV_SQI_utils;
import U_V.ESU.list.List_ESU_X_stringlistToStringArray;
import test.java.InterfaceTest.tinShell.FastCartesianIdentifyTest;
import java.io.IOException;
import java.util.HashMap;
import java.util.Iterator;
import java.util.List;
//稍后将DMA文件与内存操作替换成 jtable表内存操作 罗瑶光
public class E_pl_XA_E {
	@SuppressWarnings("unchecked")
		public static TinMap E_pl_XA(String plSearch, boolean mod,
		TinMap output, App NE) throws InterruptedException,
		IOException {

		if (null == output) {
			output = NE._I_U.outputMap;// later all in 1 and contans objectmap
		}
		output.put("firstTime", "true");
		output.put("start", "0");
		output.put("countJoins", "0");
		// 2make line
		List<String> lists = DoSplit.splitRegex(plSearch.replace(" ",
			"").replace("\n", ""), ";", "\\", "\\");
		String[] commands = List_ESU_X_stringlistToStringArray._E(
			lists);
		NE._I_U.acknowledge = null;
		String[] acknowledgeSwap = null;
```

```
for (String command : commands) {
        HashMap<String, Integer> scores = new HashMap<>();
        NE._I_U.commandAcknowledge = command;
        S_logger.Log.logger.info("" + "command" + command);
        CommandClass command_V = new CommandClass();
        // 函数注册 价值降低
        NE.app_S.workVerbalMap.command_V = command_V;
        NE.app_S.currentTinmap = output;
        // 之后整体替换下。方便command_V管理。
        command_V.commandWithoutNumerics = command.toString();
        command_V.commandAcknowledge = command.toString();
        command_V.command = command.toString();
        command_V.initSixActions(NE);
        command_V.initArabicNumber();
        // -1 计算逻辑关系根据不同的环境会有不同的组合，最优关系是频率出来的，不是
        // 关键字分析出来的。所以动态化计算是趋势。
        String commandSwap = doHumanTalkSwap(NE, command_V);
        command_V.commandSwap = commandSwap;
        List<String> commandLists = DoSplit.splitRegex(command,
                ":", "\\", "\\");
        command_V.commandLists = commandLists;
        // ESU_X swap系列 稍后并到测试文件中 -trif
        NE._I_U.acknowledge = List_ESU_X_stringlistToStringArray
                ._E(commandLists);
        command_V.acknowledge = NE._I_U.acknowledge;
        if (null != commandSwap) {// splitRegex稍后并到测试文件中 -trif
                commandLists = DoSplit.splitRegex(commandSwap, ":",
                        "\\", "\\");
                acknowledgeSwap = List_ESU_X_stringlistToStringArray
                        ._E(commandLists);
                command_V.acknowledgeSwap = acknowledgeSwap;
                S_logger.Log.logger.info("" + commandSwap);
                doAcknowledgeSwap(acknowledgeSwap, command, NE);
        }
        // -2
        String[] temp = NE._I_U.acknowledge.clone();
        // Iterator<String> iterators =
        // NE.app_S.workVerbalMap.cartesianWorkActionsRights
        // .keySet().iterator();
        Iterator<String> iterators = command_V.cartesianWorkActionsRightsSV
                .keySet().iterator();
        while (iterators.hasNext()) {
                String string = iterators.next();
                // String[] strings = string.split("_");//不包含 _ 无效,去掉此逻辑。
                S_logger.Log.logger.info("" + "400-10000001"+ string);
                if (null != string) {
                        /* loop s later */
                        int scaleRights;
                        scaleRights = command_V.cartesianWorkActionsRightsSV
```

```
                        .getInt(string);
                if (scaleRights < 12) {
                        IMV_SQI_utils.couldDoThenDo(string, temp,
                                output, NE, scores);
                }
                // later // in // pdn
        }
}
// SVO主谓宾的中文缩写。cartesianWorkActionsRights分解后SV和VO可以
// 以后形成严谨的指令句匹配规范
iterators = command_V.cartesianWorkActionsRightsVO
        .keySet().iterator();
while (iterators.hasNext()) {
        String string = iterators.next();
        // String[] strings = string.split("_");//不包含 _ 无效,去掉此逻辑。
        S_logger.Log.logger.info("" + "400-10000001"
                + string);
        if (null != string) {
                int scaleRights;
                if (command_V.cartesianWorkActionsRightsVO
                        .containsKey(string)) {
                        scaleRights = command_V.cartesianWorkActionsRightsVO
                                .getInt(string);
                } else {
                        scaleRights = 9999;
                }
                if (scaleRights < 12) {
                        S_logger.Log.logger.info(""
                                + "couldDoThenDo-2-" + string);
                        IMV_SQI_utils.couldDoThenDo(string, temp,
                                output, NE, scores);
                }
        }
}
// -3
command_V.cartesianWorkActionsRightsSV.clear();
command_V.cartesianWorkActionsPositionsSV.clear();
command_V.cartesianWorkActions_posSV.clear();
command_V.cartesianWorkActionsRightsVO.clear();
command_V.cartesianWorkActionsPositionsVO.clear();
command_V.cartesianWorkActions_posVO.clear();
command_V.unknown_map.clear();
/*
* 这样command_V的价值就出来了，clear之后再GC，双重清理，在垃圾器的优化
* 环境里会内存更加稳定。
*/
/* loop s later */
IMV_SQI_utils.couldDoThenDo(temp[0], temp, output, NE,
        scores);// later in pdn */
```

```
                if (temp[0].equals("获取临时表名")) {
                        // 稍后写入 元基花
                        output.put(temp[0], temp[1]);
                        output.put("type", temp[2]);
                }
                output.put("newCommand", temp[0]);
                Pl_XA_Command_E.P_E(temp, output, mod, NE);
                output.put("lastCommand", output.get("newCommand"));
        }
        if (null != NE._I_U.acknowledge) {
                if (output.get("start").toString().equals("1")) {
                        Pl_XA_Command_E.P_E(NE._I_U.acknowledge, output, mod,
                                NE);
                }
        }
        Pl_XA_Command_E.P_Check(output.get("newCommand").toString(),
                output, mod, NE);
        return output;
}
// 哲学 辩证学 辩论
private static boolean hasVerb(String[] acknowledgeSwap,
        int position, App NE) {
        String string = acknowledgeSwap[position];
        List<String> list = NE.app_S._A.parserMixedString(string);
        Iterator<String> iterator = list.iterator();
        while (iterator.hasNext()) {
                String stringIterator = iterator.next();
                if (NE.app_S.workVerbalMap.doMap.containsKey(
                        stringIterator)) {
                        return true;
                }
        }
        return false;
}
private static boolean hasData2DSubject(String[] acknowledgeSwap,
        int position, App NE) {
        String string = acknowledgeSwap[position];
        List<String> list = NE.app_S._A.parserMixedString(string);
        Iterator<String> iterator = list.iterator();
        while (iterator.hasNext()) {
                String stringIterator = iterator.next();
                if (NE.app_S.workVerbalMap.data2DSubjectMap.containsKey(
                        stringIterator)) {
                        return true;
                }
        }
        return false;
}
private static void doAcknowledgeSwap(String[] acknowledgeSwap,
```

```
            String command, App NE) {
            // null 将 授权 选择 进行 执行 数据 智慧 逻辑 操作 :null数据 矩阵 对象 表格 表
            boolean hasData2DSubjectBoolean = hasData2DSubject(
                        acknowledgeSwap, 0, NE);
            boolean hasVerbBoolean = hasVerb(acknowledgeSwap, 1, NE);
            if ((hasData2DSubjectBoolean/**/) && (hasVerbBoolean/**/)) {
                        doHuoQuBiaoMingSwap(command, NE);
                        return;
            }
            boolean hasData2DObjectBoolean = hasData2DSubject(
                        acknowledgeSwap, 2, NE);
            if ((hasVerbBoolean/**/) && (hasData2DObjectBoolean/**/)) {
                        doHuoQuBiaoMingSwap(command, NE);
                        return;
            }
    }
    @SuppressWarnings("unchecked")
    private static void doHuoQuBiaoMingSwap(String command, App NE) {
            Iterator<String> iterator = NE.app_S.tableNameMap.keySet()
                        .iterator();
            while (iterator.hasNext()) {
                        String string = iterator.next();
                        if (command.contains(string)) {
                                    NE._I_U.acknowledge = new String[3];
                                    NE._I_U.acknowledge[0] = "获取表名";
                                    NE._I_U.acknowledge[1] = string;
                                    NE._I_U.acknowledge[2] = "进行选择";
                                    return;
                        }
            }
    }
    @SuppressWarnings("unused")
    private static void doTinshellTalk() {
    }
    // later out to data swap api
    public static String doHumanTalkSwap(App NE,
            CommandClass command_V) {
            NE.app_S.workVerbalMap.setHumanTalkAfterNewBusinessTest(command_V, NE);
            FastCartesianIdentifyTest fastCartesianIdentifyTest = new FastCartesianIdentifyTest();
            fastCartesianIdentifyTest.getCartesianRelationShipFromHumanTalk(NE.app_S.workVerbalMap);
            Boolean findSubject = NE.app_S.workVerbalMap.findSubject(NE,command_V);
            return NE.app_S.workVerbalMap.returnBestTypeOfCommands(findSubject);
    }

    public static TinMap E_pl_XA(PL_XA_E orm, boolean b,
            TinMap output, App NE) throws InterruptedException,
            IOException {
            return E_pl_XA(orm.getPLSearch(), true, output, NE);
    }
```

```
}
**********************************************************************************************************
package test.java.InterfaceTest.tinShell;
import java.util.HashMap;
import java.util.Iterator;
import O_V.OSM.shell.CommandClass;
import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;
import S_A.SixActionMap.WorkVerbalMap;
import test.java.interfaces.test.CommonTestInition;
public class FastCartesianIdentifyTest {
	@SuppressWarnings("unused")
	public HashMap<String, String> getCartesianRelationShipFromHumanTalk(
			WorkVerbalMap workVerbalMap) {
			Iterator<String> iterators = workVerbalMap.command_V._IMV_SQI_SS
					.keySet().iterator();
			while (iterators.hasNext()) {
				Object object = iterators.next();
				String string = object.toString();
				S_logger.Log.logger.info("" + string);
				/*
				 * 找出不含有特殊符号的条件拆分，方便垃圾分类。 1 不含有与或非的string 2 3
				 */
				if (string.contains("&") || string.contains("|") || string.contains("!") || string.contains(";")
						|| string.contains("*") || string.contains(":") || string.contains(" ")) {
						/*
						 * 关于这种逻辑的写法逐char匹配，应该在更前分词的时候就过滤掉，因为
						 * 考虑到分词时候万一有符号+String的代号的组合转码条件，将造成更多
						 * 断开的碎片。
						 */
						WordFrequency wordFrequency = workVerbalMap.command_V._IMV_SQI_SS
								.getW(string);
						workVerbalMap.command_V.relationsGrammar.put(string,
								wordFrequency);
						// workVerbalMap.command_V._IMV_SQI_SS.remove(string);
						continue;
				}
				if (string.contains("*")) {

				}
				/*
				 * ASCII关系 字母关系 65~90 97~122 思考如果是其他语言呢？毫无价值。先不管。
				 */
				boolean findEnglish = false;
				findEnglish = findEnglishFromString(string);
				if (true == findEnglish) {
						WordFrequency wordFrequency = workVerbalMap.command_V._IMV_SQI_SS
								.getW(string);
						workVerbalMap.command_V.relationsEnglish.put(string,
								wordFrequency);
```

```
                        continue;
                }
                /*
                 * ASCII关系 特殊符号关系 这个最难处理，各种语言符号，各种特殊符号，各种。。later
                 */
                // to do
                WordFrequency wordFrequency = workVerbalMap.command_V._IMV_SQI_SS
                        .getW(string);
                workVerbalMap.command_V._IMV_SQI_SS_temp.put(string,
                        wordFrequency);
        }
        workVerbalMap.command_V._IMV_SQI_SS = workVerbalMap.command_V._IMV_SQI_SS_temp;
        return null;
}
private boolean findEnglishFromString(String string) {
        for (int i = 0; i < string.length(); i++) {
                if (string.charAt(i) > 64 && string.charAt(i) < 91) {
                        return true;
                }
                if (string.charAt(i) > 96 && string.charAt(i) < 123) {
                        return true;
                }
        }
        return false;
}
public HashMap<String, String> getCartesianRelationShipFromHumanTalks(
        String[] sentences) {
        return null;
}
public HashMap<String, String> getCartesianPromotionTVMFromRelationShips(
        HashMap<String, String> relationShips) {
        return null;
}
public HashMap<String, String> getUtilsOfCartesianPromotionTVM(
        HashMap<String, String> cartesianPromotion) {
        return null;
}
@SuppressWarnings("unchecked")
public static void main(String[] argv) {
        // 初始
        FastCartesianIdentifyTest fastCartesianIdentifyTest
        = new FastCartesianIdentifyTest();
        // 启动测试开始
        CommonTestInition commonTestInition = new CommonTestInition();
        commonTestInition.initEnvironment("去弹窗组件流测试");
        CommandClass command_V = new CommandClass();
        commonTestInition.NE.app_S.workVerbalMap.command_V = command_V;
        // 输入
        //String command = "条件为:和:功效|DNN搜索|功效|菜谱|4;";
```

```
String command = "" + "在输出的数据表中仅展示从第陆行到第"
 + "九" + "行的数据;";
command_V.commandWithoutNumerics = command.toString();
command_V.commandAcknowledge = command.toString();
command_V.command = command.toString();
command_V.initSixActions(commonTestInition.NE);
command_V.initArabicNumber();
// 计算
commonTestInition.NE.app_S.workVerbalMap.setHumanTalkAfterNewBusinessTest(command_V,
                commonTestInition.NE);
Iterator<String> iterators = command_V._IMV_SQI_SS_Q.keySet()
        .iterator();
while (iterators.hasNext()) {
        String string = iterators.next();
        WordFrequency WordFrequency = command_V._IMV_SQI_SS_Q.getW(string);
        command_V._IMV_SQI_SS.put(string, WordFrequency);
}
commonTestInition.NE.app_S.workVerbalMap.initEnvironment();
// 统计筛选归纳
/*
 * 这里效果就出来，去掉之前繁杂的调试逻辑。观测速度翻好几番。 later。。
 */
fastCartesianIdentifyTest.getCartesianRelationShipFromHumanTalk(
                commonTestInition.NE.app_S.workVerbalMap);
/*
 * 函数稳定后我会专门花时间分配 public private protected sync 函数方法。
 * 目前在没有冲突的情况下 整体先 public -- 罗瑶光
 */
commonTestInition.NE.app_S.workVerbalMap.relationshipsCombinationWithNoun();
commonTestInition.NE.app_S.workVerbalMap.relationshipsCombinationWithVerb();
commonTestInition.NE.app_S.workVerbalMap.relationshipsCombinationWithNounAndVerb();
commonTestInition.NE.app_S.workVerbalMap.initCartesianActions(commonTestInition.NE, command_V);
commonTestInition.NE.app_S.workVerbalMap
        .sortCartesianWorkActionsPositionSV(commonTestInition.NE,command_V);
commonTestInition.NE.app_S.workVerbalMap
        .sortCartesianWorkActionsDistanceSV(commonTestInition.NE,command_V);
commonTestInition.NE.app_S.workVerbalMap
        .sortCartesianWorkActionsPositionVO(commonTestInition.NE,command_V);
commonTestInition.NE.app_S.workVerbalMap
        .sortCartesianWorkActionsDistanceVO(commonTestInition.NE,command_V);
commonTestInition.NE.app_S.workVerbalMap.actionsNormalization(
        commonTestInition.NE, command_V);
S_logger.Log.logger.info("" + "-----笛卡尔词汇观测-----");
S_logger.Log.logger.info("" + "1-cartesianWorkA-"
        + commonTestInition.NE.app_S.workVerbalMap.verbInText.size());
Iterator<String> iteratorStrings
= commonTestInition.NE.app_S.workVerbalMap.verbInText
        .keySet().iterator();
while (iteratorStrings.hasNext()) {
```

```
            String string = iteratorStrings.next();
            S_logger.Log.logger.info("" + string + ">" + "");
        }
        S_logger.Log.logger.info("" + "1-cartesianWorkB-"
                + commonTestInition.NE.app_S.workVerbalMap.nounInText.size());
        iteratorStrings = commonTestInition.NE.app_S.workVerbalMap.nounInText
                .keySet().iterator();
        while (iteratorStrings.hasNext()) {
            String string = iteratorStrings.next();
            S_logger.Log.logger.info("" + string + ">" + "");
        }
        // 输出
        S_logger.Log.logger.info("" + "-----笛卡尔关系推荐逻辑观测-----");
        S_logger.Log.logger.info(""
                + "1-cartesianWorkActionsPositionsSV-"
                + commonTestInition.NE.app_S.workVerbalMap
                .command_V.cartesianWorkActionsPositionsSV.size());
        iteratorStrings = commonTestInition.NE.app_S.workVerbalMap
                .command_V.cartesianWorkActionsPositionsSV.keySet().iterator();
        while (iteratorStrings.hasNext()) {
            String string = iteratorStrings.next();
            S_logger.Log.logger.info("" + string + ">"
                    + commonTestInition.NE.app_S.workVerbalMap.command_V
                    .cartesianWorkActionsPositionsSV.getString(string));
        }
        S_logger.Log.logger.info(""
                + "2-cartesianWorkActionsPositionsVO-"
                + commonTestInition.NE.app_S.workVerbalMap.command_V
                .cartesianWorkActionsPositionsVO.size());
        iteratorStrings = commonTestInition.NE.app_S.workVerbalMap
                .command_V.cartesianWorkActionsPositionsVO.keySet().iterator();
        while (iteratorStrings.hasNext()) {
            String string = iteratorStrings.next();
            S_logger.Log.logger.info("" + string + ">"
                    + commonTestInition.NE.app_S.workVerbalMap.command_V
                    .cartesianWorkActionsPositionsVO.getString(string));
        }
        iteratorStrings = commonTestInition.NE.app_S.workVerbalMap
                .command_V.cartesianWorkActionsRightsSV.keySet().iterator();
        while (iteratorStrings.hasNext()) {
            String string = iteratorStrings.next();
            S_logger.Log.logger.info("" + string + ">"
                    + commonTestInition.NE.app_S.workVerbalMap.command_V
                    .cartesianWorkActionsRightsSV.getString(string));
        }//later优化

        S_logger.Log.logger.info(""
                + "4-cartesianWorkActionsRightsVO-"
                + commonTestInition.NE.app_S.workVerbalMap.command_V
```

```
                .cartesianWorkActionsRightsVO.size());
        iteratorStrings = commonTestInition.NE.app_S.workVerbalMap
                .command_V.cartesianWorkActionsRightsVO.keySet().iterator();
        while (iteratorStrings.hasNext()) {
                String string = iteratorStrings.next();
                S_logger.Log.logger.info("" + string + ">"
                        + commonTestInition.NE.app_S.workVerbalMap.command_V
                        .cartesianWorkActionsRightsVO.getString(string));
        }
        iteratorStrings = commonTestInition.NE.app_S.workVerbalMap
                .command_V.cartesianWorkActions_posSV.keySet().iterator();
        while (iteratorStrings.hasNext()) {
                String string = iteratorStrings.next();
                S_logger.Log.logger.info("" + string + ">"
                        + commonTestInition.NE.app_S.workVerbalMap.command_V
                        .cartesianWorkActions_posSV.getString(string));
        }
        iteratorStrings = commonTestInition.NE.app_S.workVerbalMap
                .command_V.cartesianWorkActions_posVO
                .keySet().iterator();
        while (iteratorStrings.hasNext()) {
                String string = iteratorStrings.next();
                S_logger.Log.logger.info("" + string + ">"
                        + commonTestInition.NE.app_S.workVerbalMap.command_V
                        .cartesianWorkActions_posVO.getString(string));
        }
        iteratorStrings = commonTestInition.NE.app_S.workVerbalMap
                .command_V.cartesianWorkActionsRightsParserSV.keySet().iterator();
        while (iteratorStrings.hasNext()) {
                String string = iteratorStrings.next();
                S_logger.Log.logger.info("" + string + ">"
                        + commonTestInition.NE.app_S.workVerbalMap.command_V
                        .cartesianWorkActionsRightsParserSV.getString(string));
        }
        iteratorStrings = commonTestInition.NE.app_S.workVerbalMap
                .command_V.cartesianWorkActionsRightsParserVO
                .keySet().iterator();
        while (iteratorStrings.hasNext()) {
                String string = iteratorStrings.next();
                S_logger.Log.logger.info("" + string + ">"
                        + commonTestInition.NE.app_S.workVerbalMap.command_V
                        .cartesianWorkActionsRightsParserVO.getString(string));
        }
        commonTestInition.endEnvironment();
    }
}
**********************************************************************************************
package S_A.SixActionMap;
import ME.VPC.M.app.App;
```

```
import O_V.OSM.shell.CommandClass;
import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;
import S_A.pheromone.IMV_SQI;
import test.java.InterfaceTest.chineseParser.DemoAfterPOSTest;
import test.java.InterfaceTest.chineseParser.DemoPOSTest;
import java.util.Iterator;
import java.util.List;
import A_V.ASQ.PSU.test.TimeCheck;
/*
 * 1 6元SDLC
 * 2 sdlc obss
 * 3 obss normalization
 * 4 normalizational format
 * 5 format map
 * 6 map parser
 * 7 parser in PDE model
 * 8 model in time norms
 * 全部基于罗瑶光著作权基础堆积即可。
 */
@SuppressWarnings("unchecked")
public class WorkVerbalMap extends WorkVerbalMap_X {
	public boolean findSubject(App NE, CommandClass command_V) {
		initEnvironment();
		// small talk calculus
		// m 一旦笛卡尔，单字组合就没有用了，仅仅依赖分词即可。
		relationshipsCombinationWithNoun();
		// d 看了计算哲学后，我才意识到我40年生命中语文功底算是白学了。
		relationshipsCombinationWithVerb();
		// md
		relationshipsCombinationWithNounAndVerb();
		// init cartesianActions
		initCartesianActions(NE, command_V);
		//SVO的关系细化分解后，逻辑操作会更加地精确。
		sortCartesianWorkActionsPositionSV(NE, command_V);
		sortCartesianWorkActionsDistanceSV(NE, command_V);
		sortCartesianWorkActionsPositionVO(NE, command_V);
		sortCartesianWorkActionsDistanceVO(NE, command_V);
		actionsNormalization(NE, command_V);
		if (!objectMap.isEmpty() && !verbMap.isEmpty()) {
			return true;
		}
		return false;
	}
	@SuppressWarnings("unused")
	public void setHumanTalkAfterNewBusinessTest(
		CommandClass command_V, App NE) {
		if (false == command_V.initedArabicNumber) {
			int res = new StudyVerbalMap().extractNumberfromString(
				command_V);
```

```
        }
        /*
         * 将replace 提取出来，然后进行字符的长度进行replace //。。。todo
         * */
        this.humanTalk = command_V.command;
        /*
        * 分词 提取 英文段和数字段形成变量。比如dnn 12345等
        */
        TimeCheck t = new TimeCheck();
        t.begin();
        command_V._IMV_SQI_SS_ = NE.app_S._A.parserMixedString(
                command_V.chineseSimpleCommandWithoutNumerics);
        t.end();
        t.duration();
        command_V._IMV_SQI_SS = NE.app_S._A.getWordFrequencyMap(
                command_V._IMV_SQI_SS_, NE);
        NE.app_S._A.initPCAWordPOS(command_V._IMV_SQI_SS, NE);
        IMV_SQI pos = NE.app_S._A.getPosCnToCn();
        DemoPOSTest demoPOSTest = new DemoPOSTest();
        /*
        *  demoPOSTest.testPOS(command_V._IMV_SQI_SS_, pos);
        *  --解决方法 首先分解
        */
        List<String> setsInput = demoPOSTest.testPOSOnlyGetList(
                command_V._IMV_SQI_SS_, pos);
        //--再list组合修正
        DemoAfterPOSTest demoAfterPOSTest = new DemoAfterPOSTest();
        List<String> setsAfterInput = demoAfterPOSTest.testAfterPOS(
                setsInput, pos);
        //--最终将修正的list 处理成 wordFrequency 格式list。
        demoPOSTest.testPOStoWordFrequencyList(setsAfterInput, pos);
        // loop update and insect
        Iterator<String> iterators = demoPOSTest.noun.keySet()
                .iterator();
        while (iterators.hasNext()) {
                String temp = iterators.next();
                if (command_V._IMV_SQI_SS.containsKey(temp)) {
                        WordFrequency wordFrequency = command_V._IMV_SQI_SS
                                .get(temp);
                        WordFrequency wordFrequencyTemp = demoPOSTest.noun
                                .get(temp);
                        wordFrequency.I_pos(wordFrequencyTemp.get_pos());
                        command_V._IMV_SQI_SS.put(temp, wordFrequency);
                }
        }
        iterators = demoPOSTest.verb.keySet().iterator();
        while (iterators.hasNext()) {
                String temp = iterators.next();
                if (command_V._IMV_SQI_SS.containsKey(temp)) {
```

```
                        WordFrequency wordFrequency = command_V._IMV_SQI_SS
                                .get(temp);
                        WordFrequency wordFrequencyTemp = demoPOSTest.verb
                                .get(temp);
                        wordFrequency.I_pos(wordFrequencyTemp.get_pos());
                        command_V._IMV_SQI_SS.put(temp, wordFrequency);
                }
        }
        iterators = demoPOSTest.adj.keySet().iterator();
        while (iterators.hasNext()) {
                String temp = iterators.next();
                if (command_V._IMV_SQI_SS.containsKey(temp)) {
                        WordFrequency wordFrequency = command_V._IMV_SQI_SS
                                .get(temp);
                        WordFrequency wordFrequencyTemp = demoPOSTest.adj.get(
                                temp);
                        wordFrequency.I_pos(wordFrequencyTemp.get_pos());
                        command_V._IMV_SQI_SS.put(temp, wordFrequency);
                }
        }
        iterators = demoPOSTest.adv.keySet().iterator();
        while (iterators.hasNext()) {
                String temp = iterators.next();
                if (command_V._IMV_SQI_SS.containsKey(temp)) {
                        WordFrequency wordFrequency = command_V._IMV_SQI_SS
                                .get(temp);
                        WordFrequency wordFrequencyTemp = demoPOSTest.adv.get(
                                temp);
                        wordFrequency.I_pos(wordFrequencyTemp.get_pos());
                        command_V._IMV_SQI_SS.put(temp, wordFrequency);
                }
        }
}
public void setHumanTalk(CommandClass command_V, App NE) {
        command_V._IMV_SQI_SS.clear();
        command_V._IMV_SQI_SS_.clear();
        command_V._IMV_SQI_S_.clear();
        this.humanTalk = command_V.command;
        // 分词 提取 英文段和数字段形成变量。比如dnn 12345等
        command_V._IMV_SQI_SS_ = NE.app_S._A.parserMixedString(
                command_V.command);
        for (int i = 0; i < command_V._IMV_SQI_SS_.size(); i++) {
                S_logger.Log.logger.info("" + command_V._IMV_SQI_SS_.get(
                        i));
        }
        command_V._IMV_SQI_SS = NE.app_S._A.getWordFrequencyMap(
                command_V._IMV_SQI_SS_, NE);
        NE.app_S._A.initPCAWordPOS(command_V._IMV_SQI_SS, NE);
}
```

```
	/*
	* 先处理仅一个主谓宾的简单长句，以后处理复杂带连词的多主宾复句子。
	* 在输出的数据表中仅展示列名为中药名称，打分和功效列这三个即可
	* 这个把被逻辑以后用指令集的形式分出去。--later
	* */
	public String returnBestTypeOfCommands(Boolean findSubject) {
		// init shortChineseActions
		Iterator<String> iterator = objectMap.keySet().iterator();
		while (iterator.hasNext()) {
			String subject = iterator.next();
			if (babeiMap.containsKey("把")) {
				if (objectMap.getInt(subject) < babeiMap.getInt("把")) {
					subjectName += subject;
				}
				if (objectMap.getInt(subject) > babeiMap.getInt("把")) {
					objectName += subject;
				}
			} else if (babeiMap.containsKey("被")) {
				if (objectMap.getInt(subject) < babeiMap.getInt("被")) {
					objectName += subject;
				}
				if (objectMap.getInt(subject) > babeiMap.getInt("被")) {
					subjectName += subject;
				}
			} else {
				if (null == subjectName) {
					subjectName += subject;
				} else {
					objectName += subject;
				}
			}
		}
		if (findSubject) {
			String output = "";
			if (null != subjectName) {
				output = subjectName;
			}
			output += ":";
			if (null != doName) {
				output += doName;
			}
			output += ":";
			if (null != objectName) {
				output += objectName;
			}
			return output;// later
		}
		return null;
	}
```

```
}
//计算辩证意识大幅减少大量算子。
**********************************************************************************
package S_A.SixActionMap;
import ME.VPC.M.app.App;
import O_V.OSM.shell.CommandClass;
import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;
import java.util.Iterator;
/* 1 6元SDLC
 * 2 sdlc obss
 * 3 obss normalization
 * 4 normalizational format
 * 5 format map
 * 6 map parser
 * 7 parser in PDE model
 * 8 model in time norms
 * 全部基于罗瑶光著作权基础堆积即可。
 *
@SuppressWarnings("unchecked") // 稍后优化新陈代谢 逻辑
public class WorkVerbalMap_X extends WorkVerbalMap_X_S {
	// 稍后去重 -trif
	public void relationshipsCombinationWithNounAndVerb() {
		Iterator<String> iteratorNounMd = nounInText.keySet()
			.iterator();
		NextNounMd: while (iteratorNounMd.hasNext()) {
			String stringNounMd = iteratorNounMd.next();
			if (stringNounMd.isEmpty()) {
				continue NextNounMd;
			}
			if (!command_V._IMV_SQI_SS.containsKey(stringNounMd)) {
				/*稍后精简条件优化变量*/
				continue NextNounMd;
			}
			WordFrequency wordFrequencyNounMd = command_V._IMV_SQI_SS
				.getW(stringNounMd);
			int averagePositionNounMd = wordFrequencyNounMd
				.getAveragePosition();

			Iterator<String> iteratorVerbNd = verbInText.keySet()
				.iterator();
			NextVerbNd: while (iteratorVerbNd.hasNext()) {
				String stringVerbNd = iteratorVerbNd.next();
				if (stringVerbNd.isEmpty()) {
					continue NextVerbNd;
				}
				if (!command_V._IMV_SQI_SS.containsKey(
					stringVerbNd)) {
					/*稍后精简条件优化变量*/
					continue NextVerbNd;
```

```
                        }
                        WordFrequency wordFrequencyVerbNd = command_V._IMV_SQI_SS
                                .getW(stringVerbNd);
                        int averagePositionVerbNd = wordFrequencyVerbNd
                                .getAveragePosition();
                        if (stringNounMd.equalsIgnoreCase(stringVerbNd)) {
                                continue NextVerbNd;
                        }
                        if (!((wordFrequencyNounMd.get_pos().contains("动")
                                || wordFrequencyNounMd.get_pos().contains("名"))
                                && (wordFrequencyNounMd.get_pos().contains("动")
                                        || wordFrequencyNounMd.get_pos().contains(
                                                "名")))) {
                                // 因为笛卡尔交集，所以需要用pos map来识别。
                                continue NextVerbNd;
                        }
                        // noun-verb
                        String rootNd = stringNounMd + stringVerbNd;
                        String rootMd = stringVerbNd + stringNounMd;
                        int rightNd = Math.abs(averagePositionNounMd
                                - averagePositionVerbNd);
                        int positionNd = (averagePositionNounMd
                                + averagePositionVerbNd) >> 1;
                        if (2 > rightNd && rootNd.length() < 3) {
                                //nounInText.remove(stringNounMd);
                                //verbInText.remove(stringVerbNd);
                                verbInText.put(rootNd, positionNd);
                                verbInText.put(rootMd, positionNd);
                                WordFrequency wordFrequency = new WordFrequency(
                                        1.0, rootNd);
                                wordFrequency.positions.add(positionNd);
                                wordFrequency.I_pos("动词名词");
                                command_V._IMV_SQI_SS.put(rootNd, wordFrequency);
                                command_V._IMV_SQI_SS.put(rootMd, wordFrequency);
                        }
                }
        }
}
public void relationshipsCombinationWithVerb() {
        Iterator<String> iteratorVerbM = verbInText.keySet()
                .iterator();
        NextVerbM: while (iteratorVerbM.hasNext()) {
                String stringVerbM = iteratorVerbM.next();
                if (stringVerbM.isEmpty()) {
                        continue NextVerbM;
                }
                if (!command_V._IMV_SQI_SS.containsKey(stringVerbM)) {
                        /*稍后精简条件优化变量*/
                        continue NextVerbM;
```

```
                }
                WordFrequency wordFrequencyVerbM = command_V._IMV_SQI_SS
                        .getW(stringVerbM);
                int averagePositionVerbM = wordFrequencyVerbM
                        .getAveragePosition();
                Iterator<String> iteratorVerbN = verbInText.keySet()
                        .iterator();
                NextVerbN: while (iteratorVerbN.hasNext()) {
                        String stringVerbN = iteratorVerbN.next();
                        if (stringVerbN.isEmpty()) {
                                continue NextVerbN;
                        }
                        if (!command_V._IMV_SQI_SS.containsKey(stringVerbN)) {
                                /*稍后精简条件优化变量*/
                                continue NextVerbN;
                        }
                        WordFrequency wordFrequencyVerbN = command_V._IMV_SQI_SS
                                .getW(stringVerbN);
                        int averagePositionVerbN = wordFrequencyVerbN
                                .getAveragePosition();
                        if (stringVerbN.equalsIgnoreCase(stringVerbN)) {
                                continue NextVerbN;
                        }
                        if (!wordFrequencyVerbM.get_pos().equals(
                                wordFrequencyVerbN.get_pos())) {
                                // 因为笛卡尔交集，所以需要用pos map来识别。
                                continue NextVerbN;
                        }
                        // noun-verb
                        String rootN = stringVerbM + stringVerbN;
                        String rootM = stringVerbN + stringVerbM;
                        int rightN = Math.abs(averagePositionVerbM
                                - averagePositionVerbN);
                        int positionN = (averagePositionVerbM
                                + averagePositionVerbN) >> 1;
                        if (2 > rightN && rootN.length() < 3) {
                                //verbInText.remove(stringVerbM);
                                //verbInText.remove(stringVerbN);
                                verbInText.put(rootN, positionN);
                                verbInText.put(rootM, positionN);
                                WordFrequency wordFrequency = new WordFrequency(
                                        1.0, rootN);
                                wordFrequency.positions.add(positionN);
                                wordFrequency.I_pos("动词");
                                command_V._IMV_SQI_SS.put(rootN, wordFrequency);
                                command_V._IMV_SQI_SS.put(rootM, wordFrequency);
                        }
                }
        }
```

```
	}
	public void relationshipsCombinationWithNoun() {
		Iterator<String> iteratorNounM = nounInText.keySet()
			.iterator();
		NextNounM: while (iteratorNounM.hasNext()) {
			String stringNounM = iteratorNounM.next();
			if (stringNounM.isEmpty()) {
				continue NextNounM;
			}
			if (!command_V._IMV_SQI_SS.containsKey(stringNounM)) {
				continue NextNounM;
			}
			WordFrequency wordFrequencyNounM = command_V._IMV_SQI_SS
				.getW(stringNounM);
			int averagePositionNounM = wordFrequencyNounM
				.getAveragePosition();
			Iterator<String> iteratorNounN = nounInText.keySet()
				.iterator();
			NextNounN: while (iteratorNounN.hasNext()) {
				String stringNounN = iteratorNounN.next();
				if (stringNounN.isEmpty()) {
					continue NextNounN;
				}
				if (!command_V._IMV_SQI_SS.containsKey(stringNounN)) {
					continue NextNounN;
				}
				WordFrequency wordFrequencyNounN = command_V._IMV_SQI_SS
					.getW(stringNounN);
				int averagePositionNounN = wordFrequencyNounN
					.getAveragePosition();
				if (stringNounM.equalsIgnoreCase(stringNounN)) {
					continue NextNounN;
				}
				if (!wordFrequencyNounM.get_pos().equals(
					wordFrequencyNounN.get_pos())) {
					// 因为笛卡尔交集，所以需要用pos map来识别。
					continue NextNounN;
				}
				// noun-verb
				String rootM = stringNounM + stringNounN;
				String rootN = stringNounN + stringNounM;
				int rightM = Math.abs(averagePositionNounM
					- averagePositionNounN);
				int positionM = (averagePositionNounM
					+ averagePositionNounN) >> 1;
				//S_logger.Log.logger.info("" + rootM + "--" + rightM);
				if (2 > rightM && rootM.length() < 3) {
					//S_logger.Log.logger.info("" + rootM + "--" + rightM);
					//nounInText.remove(stringNounM);
```

```
                                //nounInText.remove(stringNounN);
                                nounInText.put(rootM, positionM);
                                nounInText.put(rootN, positionM);
                                WordFrequency wordFrequency = new WordFrequency(
                                        1.0, rootM);
                                wordFrequency.positions.add(positionM);
                                wordFrequency.I_pos("名词");
                                command_V._IMV_SQI_SS.put(rootM, wordFrequency);
                                command_V._IMV_SQI_SS.put(rootN, wordFrequency);
                        }
                }
        }
}
// tree sort 比quick top快，但耗费大量stack，于是不考虑。
public void sortCartesianWorkActionsDistanceSV(App NE,
        CommandClass command_V) {
        actionsDistanceV_SV = new int[command_V.cartesianWorkActionsRightsSV
                .size()];
        actionsDistance_SV = new String[command_V.cartesianWorkActionsRightsSV
                .size()];
        Iterator<String> iterator = command_V.cartesianWorkActionsRightsSV
                .keySet().iterator();
        int i = 0;
        while (iterator.hasNext()) {
                String string = iterator.next();
                actionsDistance_SV[i] = string;
                actionsDistanceV_SV[i++] = command_V.cartesianWorkActionsRightsSV
                        .getInt(string);
        }
        NE.app_S.lYGSortESU9D.javaSort(actionsDistanceV_SV,
                actionsDistance_SV);
        // loop
        for (i = 0; i < actionsDistance_SV.length; i++) {
                //
        }
}
//NE.app_S.lYGSortESU9D.javaSort关于RNN精度排序之后做联想扩展功能时候会用到，先保留。
public void sortCartesianWorkActionsDistanceVO(App NE,
        CommandClass command_V) {
        actionsDistanceV_VO = new int[command_V.cartesianWorkActionsRightsVO
                .size()];
        actionsDistance_VO = new String[command_V.cartesianWorkActionsRightsVO
                .size()];
        Iterator<String> iterator = command_V.cartesianWorkActionsRightsVO
                .keySet().iterator();
        int i = 0;
        while (iterator.hasNext()) {
                String string = iterator.next();
                actionsDistance_VO[i] = string;
```

```
                actionsDistanceV_VO[i++] = command_V.cartesianWorkActionsRightsVO
                        .getInt(string);
        }
        NE.app_S.lYGSortESU9D.javaSort(actionsDistanceV_VO,
                actionsDistance_VO);
        // loop
        for (i = 0; i < actionsDistance_VO.length; i++) {
                //
        }
    }
    public void sortCartesianWorkActionsPositionSV(App NE,
        CommandClass command_V) {
        actionsPositionV_SV = new int[command_V.cartesianWorkActionsPositionsSV
                .size()];
        actionsPosition_SV = new String[command_V.cartesianWorkActionsPositionsSV
                .size()];
        Iterator<String> iterator = command_V.cartesianWorkActionsPositionsSV
                .keySet().iterator();
        int i = 0;
        while (iterator.hasNext()) {
                String string = iterator.next();
                actionsPosition_SV[i] = string;
                actionsPositionV_SV[i++] = command_V.cartesianWorkActionsPositionsSV
                        .getInt(string);
        }
        NE.app_S.lYGSortESU9D.javaSort(actionsPositionV_SV,
                actionsPosition_SV);
        // loop
        for (i = 0; i < actionsPositionV_SV.length; i++) {
                //
        }
    }
    public void sortCartesianWorkActionsPositionVO(App NE,
        CommandClass command_V) {
        actionsPositionV_VO = new int[command_V.cartesianWorkActionsPositionsVO.size()];
        actionsPosition_VO = new String[command_V.cartesianWorkActionsPositionsVO.size()];
        Iterator<String> iterator = command_V.cartesianWorkActionsPositionsVO.keySet().iterator();
        int i = 0;
        while (iterator.hasNext()) {
                String string = iterator.next();
                actionsPosition_VO[i] = string;
                actionsPositionV_VO[i++] = command_V.cartesianWorkActionsPositionsVO
                        .getInt(string);
        }
        NE.app_S.lYGSortESU9D.javaSort(actionsPositionV_VO,
                actionsPosition_VO);
        // loop
        for (i = 0; i < actionsPositionV_VO.length; i++) {
                //
```

```
                }
        }
        public void actionsNormalization(App NE, CommandClass command_V) {
        }
}
**********************************************************************************************************
package S_A.SixActionMap;
import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;
import S_A.pheromone.IMV_SQI;
import java.util.Iterator;
import java.util.LinkedList;
import ME.VPC.M.app.App;
import O_V.OSM.shell.CommandClass;
/*
 * 1 6元SDLC
 * 2 sdlc obss
 * 3 obss normalization
 * 4 normalizational format
 * 5 format map
 * 6 map parser
 * 7 parser in PDE model
 * 8 model in time norms
 * 全部基于罗瑶光著作权基础堆积即可。
 **/
@SuppressWarnings("unchecked")
public class WorkVerbalMap_X_S {
        public String doName;
        public String subjectName;
        public String objectName;
        public String humanTalk;
        public IMV_SQI fixMap;
        public IMV_SQI doMap;
        public IMV_SQI objectMap;
        public IMV_SQI babeiMap;
        public IMV_SQI verbMap;
        public IMV_SQI data2DSubjectMap;
        public IMV_SQI positionMap;
        public IMV_SQI nounInText;
        public IMV_SQI verbInText;
        public IMV_SQI nounInTextSimple;
        public IMV_SQI verbInTextSimple;
        /*
         * 开始设计高级笛卡尔关系 渡
         **/
        public IMV_SQI nounInTextReason;
        public IMV_SQI verbInTextReason;
        /*
         * 开始设计高级笛卡尔关系 簇
         **/
```

```
	public IMV_SQI nounInTextFix;
	public IMV_SQI verbInTextFix;
	/**/
	public CommandClass command_V;
	public IMV_SQI ActionsObject;
	public IMV_SQI nounInTextFull;// later
	public IMV_SQI verbInTextFull;// later
	public IMV_SQI cartesianWorkActionsFull;// later
	public LinkedList<String> shortString = new LinkedList<>();
	public int[] actionsPositionV_SV;
	public String[] actionsPosition_SV;
	public int[] actionsDistanceV_SV;
	public String[] actionsDistance_SV;
	public int[] actionsPositionV_VO;
	public String[] actionsPosition_VO;
	public int[] actionsDistanceV_VO;
	public String[] actionsDistance_VO;
	public int i = 0;
	public void initEnvironment() {
		subjectName = null;
		doName = null;
		objectName = null;
		objectMap.clear();
		verbMap.clear();
		babeiMap.clear();
		nounInText.clear();// small calculus , later do full
		verbInText.clear();
		Iterator<String> iterator = command_V._IMV_SQI_SS.keySet()
				.iterator();
		while (iterator.hasNext()) {
			String string = iterator.next();
			if (data2DSubjectMap.containsKey(string)) {
				objectMap.put(string, i++);
			}
			if (doMap.containsKey(string)) {
				doName += string;
				verbMap.put(string, i++);
			}
			if (fixMap.containsKey(string)) {
				babeiMap.put(string, i++);
			}
			// pos load
			WordFrequency wordFrequency = command_V._IMV_SQI_SS.getW(
				string);
			// 一切数据首先都应该名词化*/
			nounInText.put(string, wordFrequency);
			verbInText.put(string, wordFrequency);
			S_logger.Log.logger.info("" + wordFrequency.positions);
		}
```

```
	}
	@SuppressWarnings("unused")
	public void initCartesianActions(App NE, CommandClass command_V) {
		Iterator<String> iteratorNoun = nounInText.keySet()
			.iterator();
		NextNoun: while (iteratorNoun.hasNext()) {
			String stringNoun = iteratorNoun.next();
			if (stringNoun.isEmpty()) {
				continue NextNoun;
			}
			if (!command_V._IMV_SQI_SS.containsKey(stringNoun)) {
				continue NextNoun;
			}
			WordFrequency wordFrequencyNoun = command_V._IMV_SQI_SS
				.getW(stringNoun);
			int averagePositionNoun = wordFrequencyNoun
				.getAveragePosition();
			Iterator<String> iteratorVerb = verbInText.keySet()
				.iterator();
			NextVerb: while (iteratorVerb.hasNext()) {
				String stringVerb = iteratorVerb.next();
				if (stringVerb.isEmpty()) {
					continue NextVerb;
				}
				if (!command_V._IMV_SQI_SS.containsKey(stringVerb)) {
					continue NextVerb;
				}
				WordFrequency wordFrequencyVerb = command_V._IMV_SQI_SS
					.getW(stringVerb);
				int averagePositionVerb = wordFrequencyVerb
					.getAveragePosition();
				// noun-verb
				String root = "";
				//String root_v = "";
				String root_pos = "";
				//String root_pos_v = "";
				if (NE.app_S.studyVerbalMap.initonDelegate
					.containsKey(stringVerb)) {
					stringVerb = NE.app_S.studyVerbalMap.initonDelegate
						.getString(stringVerb);
				}
				if (averagePositionNoun < averagePositionVerb) {
					//		later do noun's domains.
					if (NE.app_S.studyVerbalMap.initonDelegate
						.containsKey(stringNoun)) {
						stringNoun = NE.app_S.studyVerbalMap.initonDelegate
							.getString(stringNoun);
					}
					if (NE.app_S.studyVerbalMap.initonDelegate
```

```
                .containsKey(stringVerb)) {
                stringVerb = NE.app_S.studyVerbalMap.initonDelegate
                        .getString(stringVerb);
            }
            root += stringNoun;
            root += "+";
            root += stringVerb;

            root_pos += "_stringNoun" + averagePositionNoun;
            root_pos += "_stringVerb" + averagePositionVerb;

            /* root_pos_v += "_stringVerb" + averagePositionVerb;
            *  root_pos_v += "_stringNoun" + averagePositionNoun;
            */
            int right = Math.abs(averagePositionNoun
                    - averagePositionVerb);
            int position = (averagePositionNoun
                    + averagePositionVerb) >> 1;
            if (!command_V.cartesianWorkActionsRightsSV
                    .containsKey(root)
                    && !command_V.cartesianWorkActionsPositionsSV
                            .containsKey(root) && right > 0) {
                    if (right < NE.app_S.initonsDistanceRelationship) {
                            if (!root.contains(" ")) {
                                    command_V.cartesianWorkActionsRightsSV
                                            .put(root, right);
                                    command_V.cartesianWorkActionsRightsParserSV.put(
                                            stringNoun + "+", right);
                                    command_V.cartesianWorkActionsRightsParserSV.put(
                                            "+" + stringVerb, right);
                            }
                    }
            }
    } else {
            if (NE.app_S.studyVerbalMap.initonDelegate
                    .containsKey(stringNoun)) {
                    stringNoun = NE.app_S.studyVerbalMap.initonDelegate
                            .getString(stringNoun);
            }
            if (NE.app_S.studyVerbalMap.initonDelegate
                    .containsKey(stringVerb)) {
                    stringVerb = NE.app_S.studyVerbalMap.initonDelegate
                            .getString(stringVerb);
            }
            root += stringVerb;
            root += "-";
            root += stringNoun;
            root_pos += "_stringVerb" + averagePositionVerb;
            root_pos += "_stringNoun" + averagePositionNoun;
```

```
                                int right = Math.abs(averagePositionNoun
                                        - averagePositionVerb);
                                int position = (averagePositionNoun
                                        + averagePositionVerb) >> 1;
                                if (!command_V.cartesianWorkActionsRightsVO
                                        .containsKey(root)
                                        && !command_V.cartesianWorkActionsPositionsVO
                                                .containsKey(root) && right > 0) {
                                        if (right < NE.app_S.initonsDistanceRelationship) {
                                                if (!root.contains(" ")) {
                                                        command_V.cartesianWorkActionsRightsVO
                                                                .put(root, right);
                                                        command_V.cartesianWorkActionsRightsParserVO.put(
                                                                stringVerb + "-", right);
                                                        command_V.cartesianWorkActionsRightsParserVO.put(
                                                                "-" + stringNoun, right);
                                                }
                                        }
                                }
                        }
                }
        }
}
public void initActionMap(CommandClass command_V) {
        // 计算关机分层
        objectMap = new IMV_SQI();
        babeiMap = new IMV_SQI();
        verbMap = new IMV_SQI();
        fixMap = new IMV_SQI();
        nounInText = new IMV_SQI();
        verbInText = new IMV_SQI();
        // （首-先，一，开始，于是，顺其自然，）
        // （将，获-取-得，授权，选择，确-定-保，认-准-定，标-记-出，拿-出-到-来，把，）
        data2DSubjectMap = new IMV_SQI();data2DSubjectMap.put("表", true);
        data2DSubjectMap.put("表格", true);data2DSubjectMap.put("表单", true);
        data2DSubjectMap.put("东西", true);data2DSubjectMap.put("物", true);
        data2DSubjectMap.put("表库", true);data2DSubjectMap.put("矩阵", true);
        data2DSubjectMap.put("文档", true);data2DSubjectMap.put("文件", true);
        data2DSubjectMap.put("对象", true);data2DSubjectMap.put("单子", true);
        data2DSubjectMap.put("数据库", true);data2DSubjectMap.put("数据", true);
        data2DSubjectMap.put("文章", true);data2DSubjectMap.put("文献", true);
        // （表 表格-单-库，矩阵，文-档-件，对象）
        // 这里有很多问题，最终表达会影响计算精度，以后会细化分类逐步优化。-trif
        doMap = new IMV_SQI();doMap.put("进行", true);      doMap.put("执行", true);
        doMap.put("提", true);doMap.put("操", true);doMap.put("跟进", true);   doMap.put("更近", true);
        doMap.put("更进", true);          doMap.put("数据", true);          doMap.put("智慧", true);
        doMap.put("逻辑", true);doMap.put("选择", true);          doMap.put("操作", true);
        doMap.put("点击", true);doMap.put("点", true); doMap.put("确认", true);
        doMap.put("做", true);  doMap.put("将", true);  doMap.put("获取", true);
```

```
                doMap.put("获", true);  doMap.put("获得", true);doMap.put("取得", true);
                doMap.put("取", true);  doMap.put("授权", true);doMap.put("授", true);
                doMap.put("选", true);doMap.put("确定", true); doMap.put("确保", true);
                doMap.put("认准", true);doMap.put("认", true); doMap.put("认定", true);
                doMap.put("定", true);  doMap.put("标记", true);doMap.put("标出", true);
                doMap.put("标", true);  doMap.put("拿出", true);doMap.put("拿到", true);
                doMap.put("拿来", true);doMap.put("拿", true); fixMap.put("把", true);
                fixMap.put("被", true);  doMap.put("锁定", true);doMap.put("锁存", true);
                doMap.put("锁", true);  doMap.put("存", true);
                // （进行 执行 跟进 更近 更进 数据 智慧 逻辑 选择 操作 确认）
        }
}
```

****

2 分词前歧义中文混合数字识别模块和

--主要应用在tinshell的TVM命令句中数字识别和分词中切词前的数字识别。

--识别逻辑

--StudyVerbalMap

****

```
package S_A.SixActionMap;
import java.util.Iterator;
import O_V.OSM.shell.CommandClass;
import S_A.AVQ.OVQ.OSQ.VSQ.obj.WordFrequency;
import S_logger.Log;
public class StudyVerbalMap extends StudyVerbalMap_Q {
        public String filterString = "";
        /*
         * 这个函数用于command_V的commandString中提取出数字特征字符到map中，然后遍历map
         * 将这些特征字符的词组进行过滤掉。
         */
        public int extractNumberfromString(CommandClass command_V) {
                if (null == command_V.command) {
                        return -1;
                }
                if (null == command_V.command) {
                        return -1;
                }
                filterString = "";//修正
                getNumericsFromUnknownMapAndFiltCommand(command_V);
                //
                command_V.chineseSimpleCommandWithoutNumerics = filterString;
                command_V.initedArabicNumber = true;
                return 0;
        }
        @SuppressWarnings("unchecked")
        public void getNumericsFromUnknownMapAndFiltCommand(
                CommandClass command_V) {
                // number extra
                String string = "";
                String inputString = command_V.command;
```

```
            S_logger.Log.logger.info("" + inputString);
            inputString = command_V.simpleChineseNumberSwap(inputString);
            /*
             * 思考--涉及拆嵌套的逻辑如何分关系。
             * */
            if (command_V.chineseSimpleCommandWithoutNumerics.isEmpty()) {
                    command_V.chineseSimpleCommandWithoutNumerics = inputString
                            .toString();
            }
            String temp = "chineseSimpleCommandWithoutNumerics400-1-->"
                    + command_V.chineseSimpleCommandWithoutNumerics;
            S_logger.Log.logger.info(temp);
            int fixOrder = 0;
            for (fixOrder = 0; fixOrder < inputString
                    .length(); fixOrder++) {
                    if (!(inputString.charAt(fixOrder) <= 47 || inputString
                            .charAt(fixOrder) >= 58) || chineseNumberSets
                                    .containsKey("" + inputString.charAt(fixOrder))) {
                            string += inputString.charAt(fixOrder);
                            continue;
                    }
                    if (!string.isEmpty()) {
                            S_logger.Log.logger.info("" + string + "--"
                                    + fixOrder);
                            command_V.numericsFromUnknownString.put(string
                                    .toString(), fixOrder);
                            String output = formatNumeric(string, command_V);
                            if (null != output) {
                                    filterString += output;
                            }
                            string = "";
                    }
                    filterString += inputString.charAt(fixOrder);
                    continue;
            }
            if (!string.isEmpty()) {
                    S_logger.Log.logger.info("" + "nums-->" + string + "--"
                            + fixOrder);
                    // println函数走图形打印机，并发工程记得注释掉或者用其他的classic观测API
                    command_V.numericsFromUnknownString.put(string.toString(),
                            fixOrder);
                    string = "";
            }
            // chinese number extra
            /*
             * 之后这类输出统一归纳为频率 和时间统计函数集，方便高频优先意识。 --罗瑶光
             * */
    }
    @SuppressWarnings("unchecked")
```

```
public void formatNumericMap(CommandClass command_V) {
        Iterator<String> iterators = command_V.numericsFromUnknownString
                .keySet().iterator();
        while (iterators.hasNext()) {
                String string = iterators.next();
                formatNumeric(string, command_V);
        }
}

@SuppressWarnings("unused")
public String formatNumeric(String input,
        CommandClass command_V) {
        String string = input;
        String symbolsSwapNumericsByLength = "";
        for (int i = 0; i < string.length(); i++) {
                symbolsSwapNumericsByLength += command_V.symbolSwapNumerics;
        }
        S_logger.Log.logger.info("" + "混合数字字符探索-->" + string);
        boolean hasNumerics = false;
        boolean hasChars = false;
        if (string.contains("0") || string.contains("1") || string.contains("2") || string.contains("3")
                || string.contains("4") || string.contains("5") || string.contains("6")
                || string.contains("7") || string.contains("8") || string.contains("9")) {
                hasNumerics = true;
        }
        if (string.contains("零") || string.contains("一") || string.contains("二") || string.contains("三")
                || string.contains("四") || string.contains("五") || string.contains("六")
                || string.contains("七") || string.contains("八") || string.contains("九") || string.contains("十")
                || string.contains("十") || string.contains("百") || string.contains("千") || string.contains("万")
                || string.contains("亿")) {
                hasChars = true;
        }
        if (!hasNumerics && !hasChars) {
                //流水阀门上提优化。
                return null;
        }
        if (hasNumerics && !hasChars) {
                /*
                 * 翻译阿拉伯数字 逻辑是
                 * number = studyVerbalMap_Q.getChineseFromNumerics(number);
                 * */
                StringBuilder stringBuilder = new StringBuilder(string);
                stringBuilder = getChineseFromNumerics(stringBuilder);
                //稍后逐步替换。stringBuilder
                S_logger.Log.logger.info("" + "stringSwaped-400-1->"
                        + stringBuilder.toString());
                String stringSwaped = command_V.fasterChineseNumberSwap(
                        stringBuilder);
                S_logger.Log.logger.info("" + "stringSwaped-400-2->"
```

```
                    + stringSwaped);
            //String stringSwaped = command_V.fasterChineseNumberSwap(string);
            WordFrequency wordFrequency = new WordFrequency(1,
                    stringSwaped);
            String temp = command_V.numericsFromUnknownString
                    .getString(string);
            int tempInt = Integer.valueOf(temp);
            S_logger.Log.logger.info("" + "position-->" + tempInt);
            wordFrequency.positions.add(tempInt);
            //wordFrequency.I_frequency(1);
            wordFrequency.I_pos("变换数字字符串代词名词");
            /*
             * replace应该分离出来，作为全局任务，避免出现2*18这种错误归纳。
             * */
            command_V._IMV_SQI_SS_Q.put(stringSwaped, wordFrequency);
            command_V._IMV_SQI_SS_Q_.add(stringSwaped);
            return stringSwaped;//减少遍历
        }
        if (!hasNumerics && hasChars) {
            /* 翻译汉字数字 逻辑是
             * commandClass.fasterChineseNumberSwap(number);
             * 之后要归纳下，避免重含，如果有逆向递归操作容易死循环。*/
            StringBuilder stringBuilder = new StringBuilder(string);
            String stringSwaped = command_V.fasterChineseNumberSwap(
                    stringBuilder);
            Log.logger.info("" + "stringSwaped-400-2->"
                    + stringSwaped);
            WordFrequency wordFrequency = new WordFrequency(1,
                    stringSwaped);
            String temp = command_V.numericsFromUnknownString
                    .getString(string);
            int tempInt = Integer.valueOf(temp);
            S_logger.Log.logger.info("" + "position-->" + tempInt);
            wordFrequency.positions.add(tempInt);
            //wordFrequency.I_frequency(1);
            wordFrequency.I_pos("变换数字字符串代词名词");
            command_V._IMV_SQI_SS_Q.put(stringSwaped, wordFrequency);
            command_V._IMV_SQI_SS_Q_.add(stringSwaped);
            /*
             * 下面注释的逻辑需要按照字符串的长短排序后按照长优先进行replace，不然会
             * 字符串断层错误。
             * */
            return stringSwaped;//减少遍历
        }
        S_logger.Log.logger.info("" + "混合数字字符预处理锁定-->" + string);
        String chineseNumber = "";
        String arabicNumber = "";
        for (int i = 0; i < string.length(); i++) {
            if (string.charAt(i) > 47 && string.charAt(i) < 58) {
```

```
                arabicNumber += string.charAt(i);
                continue;
            }
            if (!arabicNumber.isEmpty()) {
                StringBuilder stringBuilder = new StringBuilder(arabicNumber);
                stringBuilder = getChineseFromNumerics(stringBuilder);
                String chineseNumberPars = stringBuilder.toString();
                String fix = "";
                S_logger.Log.logger.info("" + "chineseNumber-->"+ chineseNumber);
                S_logger.Log.logger.info(""+ "chineseNumberPars length-->"
                        + chineseNumberPars.length());
                if (!chineseNumber.isEmpty() && arabicNumber
                        .length() < 4) {
                    //稍后用StringBuilder替换加速。
                    if(chineseNumber.charAt(chineseNumber.length() - 1) != '零') {
                        fix = "零";
                    }
                }
                chineseNumber += fix + chineseNumberPars;
                arabicNumber = "";
            }
            chineseNumber += string.charAt(i);
        }
        if (!arabicNumber.isEmpty()) {
            StringBuilder stringBuilder = new StringBuilder(arabicNumber);
            stringBuilder = getChineseFromNumerics(stringBuilder);
            String chineseNumberPars=stringBuilder.toString();
            String fix = "";
            S_logger.Log.logger.info("" + "chineseNumber-->"+ chineseNumber);
            S_logger.Log.logger.info("" + "chineseNumberPars length-->" + chineseNumberPars.length());
            if (!chineseNumber.isEmpty() && arabicNumber.length() < 4) {
                if(chineseNumber.charAt(chineseNumber.length() - 1) != '零') {
                    fix = "零";
                }
            }
            chineseNumber += fix + chineseNumberPars;
            arabicNumber = "";
        }
        // fix filter
        // 稍后分离出去。
        StringBuilder stringBuilder = new StringBuilder(chineseNumber);
        stringBuilder = prefixOptimization(stringBuilder);
        // oder-fix
        chineseNumber = orderfixOptimization(chineseNumber);
        S_logger.Log.logger.info("" + "混合数字字符预处理结果-->"
                + chineseNumber);
        /* 稍后去重
         * --思考关于position的位置添加方式，是字符串的头字所取位置，那么之后分词的
         * 位置也要符合这个规范，不然有误差。或导致精度过滤的条件类计算不准确。
```

```
                * later -trif。 罗瑶光
                * */
                StringBuilder stringBuilderChineseNumber = new StringBuilder(chineseNumber);
                String regArabicNumber = command_V.fasterChineseNumberSwap(
                                stringBuilderChineseNumber);
                S_logger.Log.logger.info("" + "stringSwaped-400-2->"
                                + regArabicNumber);
                WordFrequency wordFrequency = new WordFrequency(1,
                                regArabicNumber);
                String temp = command_V.numericsFromUnknownString.getString(
                                string);
                int tempInt = Integer.valueOf(temp);
                S_logger.Log.logger.info("" + "position-->" + tempInt);
                wordFrequency.positions.add(tempInt);
                //wordFrequency.I_frequency(1);
                wordFrequency.I_pos("变换数字字符串代词名词");
                command_V._IMV_SQI_SS_Q.put(regArabicNumber, wordFrequency);
                command_V._IMV_SQI_SS_Q_.add(regArabicNumber);
                return regArabicNumber;//减少遍历
        }

    --StudyVerbalMap_Q
**********************************************************************************************************
package S_A.SixActionMap;
import O_V.OSM.shell.CommandClass;
public class StudyVerbalMap_Q extends StudyVerbalMap_X {
                @SuppressWarnings("unused")
        public StringBuilder getChineseFromNumerics(StringBuilder chars) {
                S_logger.Log.logger.info("" + chars.toString());
                //char[] chars = number.toCharArray();
                String[] chineseParser = new String[4];
                String[] chineseParsered = new String[4];
                for (int i = 0; i < chineseParser.length; i++) {
                                chineseParser[i] = "";
                                chineseParsered[i] = "";
                }
                /*
                 * 1 number swap to numberParser
                 */
                int order = 0;
                for (int right = chars.length() - 1; right >= 0; right--) {
                                char temp = chars.charAt(right);
                                if (order < 4) {
                                                chineseParser[0] = temp + chineseParser[0];
                                } else if (order < 8) {
                                                chineseParser[1] = temp + chineseParser[1];
                                } else if (order < 12) {
                                                chineseParser[2] = temp + chineseParser[2];
                                } else if (order < 16) {
```

```
                    chineseParser[3] = temp + chineseParser[3];
            }
            order++;
    }
    for (int i = 0; i < chineseParser.length; i++) {
            // S_logger.Log.logger.info("" + i + "-->" + chineseParser[i]);
    }
    /*
     * 2 numberParser swap to chineseParser
     */
    for (int i = 0; i < chineseParser.length; i++) {
            if (chineseParser[i].isEmpty()) {
                    continue;
            }
            chineseParsered[i] = doSwapUnderTenThousands(
                    chineseParser[i]);
            // S_logger.Log.logger.info("" + i + "-->" + chineseParsered[i]);
    }
    /*
     * 3 chineseParser combination
     */
    StringBuilder output = new StringBuilder();
    boolean has3 = false;
    boolean has2 = false;
    boolean has1 = false;
    boolean has0 = false;
    if (chineseParsered[3].isEmpty()) {
            has3 = false;
    } else {
            has3 = true;
    }
    output.append(chineseParsered[3]);
    if (true == has3 && !chineseParsered[3].equals("零")) {
            output.append('万');
    }
    //
    if (chineseParsered[2].isEmpty()) {
            has2 = false;
    } else {
            has2 = true;
    }
    output.append(chineseParsered[2]);
    if ((true == has2 || true == has3) && !chineseParsered[2].equals("零")) {
            output.append('亿');
    }
    //
    if (chineseParsered[1].isEmpty()) {
            has1 = false;
    } else {
```

```
                has1 = true;
        }
        output.append(chineseParsered[1]);
        if ((true == has1) && !chineseParsered[1].equals("零")) {
                output.append('万');
        }
        //
        if (chineseParsered[0].isEmpty()) {
                has0 = false;
        } else {
                if (chineseParsered[0].length() < 4) {
                        if (0 != output.length()) {
                                if (output.charAt(output.length() - 1) == '亿' || output.charAt(output.length()
                                                - 1) == '万') {
                                        output = output.append('零');
                                }
                        }
                }
                has0 = true;
        }
        output.append(chineseParsered[0]);
        /* 开头与结尾零过滤,因为正则在不同的系统中有不同的语法格式如PCRE
         * 所以java系统我手写一份。 --罗瑶光 */
        //String outputFinal = "";
        output = prefixOptimization(output);
        // oder-fix
        String outputString = orderfixOptimization(output.toString());
        S_logger.Log.logger.info("" + "output-->" + outputString);
        return new StringBuilder(outputString);
}
/* fix filter 和 oder-fix稍后提取成函数，避免重复，然后command class继承这些中间变量， 处理好哲学
 * 关系 保持简洁计算性能。。 --罗瑶光 */
public StringBuilder prefixOptimization(StringBuilder input) {
        if (0 != input.length()) {
                while (input.charAt(0) == '零' && input.length() > 1) {
                        input.deleteCharAt(0);
                        //.subSequence(1, input.length());
                }
        }
        return input;
}
public String orderfixOptimization(String input) {
        if (0 != input.length()) {
                while (input.charAt(input.length() - 1) == '零' && input
                                .length() > 1) {
                        input = input.substring(0, input.length() - 1);
                }
        }
        return input;
```

```
	}
	/* 这个逻辑开始思考千位变换， 1- 首先string 进行 swap to chars
	 * 2- 4位的 char 对应个十百千 这是单位， 3- char是0-9 需要量词翻译。
	 * 4- 翻译后加单位进行组合，量词是零需要过滤所在的单位，因为一开始是高位满足法拆分，
	 * 5- 所以开头是0，也要保留零，
	 * 6- 末尾是零需要过滤， */
	@SuppressWarnings("unused")
	public String doSwapUnderTenThousands(String string) {
		String stringChinese = "";
		String stringChineseUnits = "";
		String stringChineseUnitsFix = "";
		String stringChineseUnitsFixFlter = "";
		String output = "";
		char[] chars = string.toCharArray();
		for (int right = chars.length - 1; right >= 0; right--) {
			char temp = chars[right];
			if (temp == '0') {stringChinese = "零" + stringChinese;}
			if (temp == '1') {stringChinese = "一" + stringChinese;}
			if (temp == '2') {stringChinese = "二" + stringChinese;}
			if (temp == '3') {stringChinese = "三" + stringChinese;}
			if (temp == '4') {stringChinese = "四" + stringChinese;}
			if (temp == '5') {stringChinese = "五" + stringChinese;}
			if (temp == '6') {stringChinese = "六" + stringChinese;}
			if (temp == '7') {stringChinese = "七" + stringChinese;}
			if (temp == '8') {stringChinese = "八" + stringChinese;}
			if (temp == '9') {stringChinese = "九" + stringChinese;}
		}
		/*
		 * 加单元
		 */
		int order = 0;
		for (int i = stringChinese.length() - 1; i >= 0; i--) {
			stringChineseUnits = "" + stringChinese.charAt(i);
			if (1 == order && stringChinese.charAt(i) != '零') {
				stringChineseUnits = stringChinese.charAt(i) + "十";
			}
			if (2 == order && stringChinese.charAt(i) != '零') {
				stringChineseUnits = stringChinese.charAt(i) + "百";
			}
			if (3 == order && stringChinese.charAt(i) != '零') {
				stringChineseUnits = stringChinese.charAt(i) + "千";
			}
			stringChineseUnitsFix = stringChineseUnits
				+ stringChineseUnitsFix;
			order++;
		}
		/*
		 * 过滤零单元
		 */
```

```
                if (stringChineseUnitsFix.contains("零零")) {
                        stringChineseUnitsFix = stringChineseUnitsFix.replace("零零零零", "");
                        stringChineseUnitsFix = stringChineseUnitsFix.replace("零零零", "零");
                        stringChineseUnitsFix = stringChineseUnitsFix.replace("零零", "零");
                }
                stringChineseUnitsFix = orderfixOptimization(
                        stringChineseUnitsFix);
                return stringChineseUnitsFix;
        }

        public static void main(String[] argv) {
//十六大数测试，覆盖率100%
//                for (int i = 0; i < 99999999; i++) {
//                        for (int j = 0; j < 99999999; j++) {
//                                StringBuilder bignumber = new StringBuilder(i + ""
//                                        + j);
//                                bignumber = studyVerbalMap_Q.getChineseFromNumerics(
//                                        bignumber);
//                                commandClass.fasterChineseNumberSwap(bignumber);
//                        }
//                }
                // 不断加不断修正细化即可
        }
}
//不断加测试函数不断修正细化即可
//输出
```

--StudyVerbalMap_X

```
**********************************************************************************************************
package S_A.SixActionMap;
import ME.VPC.M.app.App;
import S_A.pheromone.IMV_SQI;
import java.lang.reflect.Field;
import java.util.HashMap;
import java.util.Map;
public class StudyVerbalMap_X {
   public IMV_SQI _SMV = new IMV_SQI();
   public IMV_SQI _SMQ = new IMV_SQI();
   public IMV_SQI _SMI = new IMV_SQI();
   public IMV_SQI initonDelegate = new IMV_SQI();
   public Map<String, Boolean> chineseNumberSets = new HashMap<>();
   public void initChineseNumberSets() {
        chineseNumberSets.put( "零" , true);chineseNumberSets.put( "一" , true);chineseNumberSets.put( "二" , true);
        chineseNumberSets.put( "三" , true);chineseNumberSets.put( "四" , true);chineseNumberSets.put( "五" , true);
        chineseNumberSets.put( "六" , true);chineseNumberSets.put( "七" , true);chineseNumberSets.put( "八" , true);
        chineseNumberSets.put( "九" , true);chineseNumberSets.put( "十" , true);chineseNumberSets.put( "百" , true);
        chineseNumberSets.put( "千" , true);chineseNumberSets.put( "万" , true);chineseNumberSets.put( "亿" , true);
        chineseNumberSets.put( "壹" , true);chineseNumberSets.put( "贰" , true);chineseNumberSets.put( "两" , true);
        chineseNumberSets.put( "叁" , true);chineseNumberSets.put( "肆" , true);chineseNumberSets.put( "伍" , true);
```

```
        chineseNumberSets.put( "陆" , true);chineseNumberSets.put( "柒" , true);chineseNumberSets.put( "捌" , true);
        chineseNumberSets.put( "玖" , true);chineseNumberSets.put( "拾" , true);chineseNumberSets.put( "佰" , true);
        chineseNumberSets.put( "仟" , true);
    }
    @SuppressWarnings("unchecked")
        public void initInitonDelegate() {
      // A
      initonDelegate.put("分析", "A"); initonDelegate.put("细化", "A"); initonDelegate.put("理解", "A");
      initonDelegate.put("吸收", "A"); initonDelegate.put("吃透", "A"); initonDelegate.put("消化", "A");
      initonDelegate.put("归纳", "A");initonDelegate.put("定义", "A"); initonDelegate.put("采集", "A");
      initonDelegate.put("分类", "A"); initonDelegate.put("统计", "A"); initonDelegate.put("计算", "A");
      initonDelegate.put("了解", "A"); initonDelegate.put("观望", "A");initonDelegate.put("统筹", "A");
      initonDelegate.put("挖掘", "A"); initonDelegate.put("联系", "A"); initonDelegate.put("关联", "A");
      initonDelegate.put("梳理", "A");
      // O
      initonDelegate.put("操作", "O");  initonDelegate.put("运作", "O"); initonDelegate.put("摆设", "O");
      initonDelegate.put("摆布", "O");  initonDelegate.put("主持", "O");  initonDelegate.put("操心", "O");
      initonDelegate.put("作业", "O");  initonDelegate.put("熟悉", "O");   initonDelegate.put("操劳", "O");
      initonDelegate.put("接触", "O");   initonDelegate.put("摸透", "O");   initonDelegate.put("触碰", "O");
      initonDelegate.put("练习", "O");  initonDelegate.put("操练", "O");
      // P
      initonDelegate.put("处理", "P"); initonDelegate.put("处置", "P"); initonDelegate.put("解决", "P");
      initonDelegate.put("运行", "P");  initonDelegate.put("下达", "P");    initonDelegate.put("收纳", "P");
      initonDelegate.put("容纳", "P");  initonDelegate.put("执行", "P");  initonDelegate.put("结果", "P");
      initonDelegate.put("处理", "P"); initonDelegate.put("落实", "P");  initonDelegate.put("成果", "P");
      initonDelegate.put("贡献", "P");  initonDelegate.put("结论", "P");   initonDelegate.put("观点", "P");
      initonDelegate.put("论文", "P"); initonDelegate.put("命令", "P");  initonDelegate.put("指示", "P");
      initonDelegate.put("指令", "P");  initonDelegate.put("报应", "P");
      initonDelegate.put("处决", "P");
      // M
      initonDelegate.put("管理", "M");  initonDelegate.put("支配", "M");  initonDelegate.put("调剂", "M");
      initonDelegate.put("整理", "M");  initonDelegate.put("排列", "M");    initonDelegate.put("序列", "M");
      initonDelegate.put("区间", "M");   initonDelegate.put("归一", "M");   initonDelegate.put("调理", "M");
      initonDelegate.put("整顿", "M");   initonDelegate.put("文档", "M");//动词用
      initonDelegate.put("档案", "M");    initonDelegate.put("分管", "M");  initonDelegate.put("运维", "M");
      initonDelegate.put("维护", "M"); initonDelegate.put("保养", "M");   initonDelegate.put("协调", "M");
      // V
      initonDelegate.put("展示", "V");  initonDelegate.put("感知", "V");   initonDelegate.put("感觉", "V");
      initonDelegate.put("感应", "V");   initonDelegate.put("敏感", "V");   initonDelegate.put("过敏", "V");
      initonDelegate.put("直觉", "V");   initonDelegate.put("知觉", "V");    initonDelegate.put("察觉", "V");
      initonDelegate.put("应激", "V");   initonDelegate.put("观察", "V");   initonDelegate.put("视觉", "V");
      initonDelegate.put("接触", "V");   initonDelegate.put("听觉", "V");    initonDelegate.put("味觉", "V");
      initonDelegate.put("嗅觉", "V");     initonDelegate.put("触觉", "V");      initonDelegate.put("感受", "V");
      initonDelegate.put("存在", "V");
      // E
      initonDelegate.put("执行", "E");   initonDelegate.put("运行", "E");//later 去重
      initonDelegate.put("做", "E");     initonDelegate.put("动", "E");   initonDelegate.put("走", "E");
      //initonDelegate.put("~", "E");//各类动词~
      initonDelegate.put("进行", "E");   initonDelegate.put("执行", "E");  initonDelegate.put("提", "E");
```

```
    initonDelegate.put("操", "E");  initonDelegate.put("跟进", "E"); initonDelegate.put("更近", "E");
    initonDelegate.put("更进", "E");  initonDelegate.put("数据", "E");
    initonDelegate.put("智慧", "E"); initonDelegate.put("逻辑", "E"); initonDelegate.put("选择", "E");
    initonDelegate.put("操作", "E");  initonDelegate.put("点击", "E");  initonDelegate.put("点", "E");
    initonDelegate.put("确认", "E");  initonDelegate.put("做", "E");  initonDelegate.put("将", "E");
    initonDelegate.put("获取", "E");  initonDelegate.put("获", "E");  initonDelegate.put("获得", "E");
    initonDelegate.put("取得", "E");  initonDelegate.put("取", "E");  initonDelegate.put("授权", "E");
    initonDelegate.put("授", "E");  initonDelegate.put("选", "E");  initonDelegate.put("确定", "E");
    initonDelegate.put("确保", "E");  initonDelegate.put("认准", "E");  initonDelegate.put("认", "E");
    initonDelegate.put("认定", "E");  initonDelegate.put("定", "E");  initonDelegate.put("标记", "E");
    initonDelegate.put("标出", "E");  initonDelegate.put("标", "E");  initonDelegate.put("拿出", "E");
    initonDelegate.put("拿到", "E");  initonDelegate.put("拿来", "E");  initonDelegate.put("拿", "E");
    initonDelegate.put("锁定", "E");//later去重 C
    initonDelegate.put("锁存", "E");  initonDelegate.put("锁", "E");  initonDelegate.put("存", "E");
    // C
    initonDelegate.put("控制", "C");  initonDelegate.put("把控", "C");  initonDelegate.put("锁定", "C");
    initonDelegate.put("登记", "C");  initonDelegate.put("制约", "C");  initonDelegate.put("管控", "C");
    initonDelegate.put("抓取", "C");  initonDelegate.put("盯紧", "C");  initonDelegate.put("控", "C");
//。。。
    // S-Noun
//  initonDelegate.put("静态", "N"); initonDelegate.put("表", "N");
//  initonDelegate.put("表格", "N");//  initonDelegate.put("表单", "N");//  initonDelegate.put("东西", "N");
//  initonDelegate.put("物", "N");//  initonDelegate.put("表库", "N");//  initonDelegate.put("矩阵", "N");
//  initonDelegate.put("文档", "N");//  initonDelegate.put("文件", "N");//  initonDelegate.put("对象", "N");
//  initonDelegate.put("单子", "N");//  initonDelegate.put("数据库", "N");//  initonDelegate.put("数据", "N");
//  initonDelegate.put("文章", "N");//  initonDelegate.put("内存", "N");//  initonDelegate.put("变量", "N");
//  initonDelegate.put("图片", "N");//  initonDelegate.put("声音", "N");//  initonDelegate.put("媒体", "N");
//  initonDelegate.put("电影", "N");//  initonDelegate.put("网页", "N");//  initonDelegate.put("程序", "N");
//  initonDelegate.put("文件夹", "N");//  initonDelegate.put("字符", "N");//  initonDelegate.put("数字", "N");
//  initonDelegate.put("数组", "N");//  initonDelegate.put("对象", "N");//  initonDelegate.put("链表", "N");
//  initonDelegate.put("树", "N");//  initonDelegate.put("插件", "N");//  initonDelegate.put("函数", "N");
//  initonDelegate.put("类", "N");//  initonDelegate.put("map", "N");//  initonDelegate.put("list", "N");
//  initonDelegate.put("vector", "N");
    // I
    initonDelegate.put("增加", "I");  initonDelegate.put("增", "I");  initonDelegate.put("加", "I");
    initonDelegate.put("生", "I");  initonDelegate.put("添加", "I");  initonDelegate.put("创造", "I");
    initonDelegate.put("new", "I");  initonDelegate.put("添", "I");  initonDelegate.put("得到", "I");
    initonDelegate.put("开启", "I");  initonDelegate.put("初始", "I");  initonDelegate.put("打开", "I");
    initonDelegate.put("启动", "I");  initonDelegate.put("收纳", "I");//later duplicate
    initonDelegate.put("获得", "I");//。。
    initonDelegate.put("获取", "I");  initonDelegate.put("收留", "I");  initonDelegate.put("酸", "I");
    initonDelegate.put("收容", "I");  initonDelegate.put("收取", "I");  initonDelegate.put("取得", "I");
    initonDelegate.put("采纳", "I");  initonDelegate.put("收购", "I");  initonDelegate.put("购买", "I");
    initonDelegate.put("采购", "I");  initonDelegate.put("迎接", "I");  initonDelegate.put("纳畜", "I");
    initonDelegate.put("添丁", "I");  initonDelegate.put("中奖", "I");  initonDelegate.put("接受", "I");
    initonDelegate.put("接纳", "I");  initonDelegate.put("收纳", "I");  initonDelegate.put("受贿", "I");
    initonDelegate.put("纳娶", "I");  initonDelegate.put("增加", "I");
    // D //危险词汇
    initonDelegate.put("删除", "D");initonDelegate.put("减少", "D");  initonDelegate.put("剔除", "D");
```

```
    initonDelegate.put("开除", "D");      initonDelegate.put("删除", "D");      initonDelegate.put("去除", "D");
    initonDelegate.put("删", "D");      initonDelegate.put("减", "D");      initonDelegate.put("省略", "D");
    initonDelegate.put("碱", "D");      initonDelegate.put("掉", "D");      initonDelegate.put("死", "D");
    initonDelegate.put("去", "D");      initonDelegate.put("开掉", "D");      initonDelegate.put("完", "D");
    initonDelegate.put("滚", "D");      initonDelegate.put("消失", "D");      initonDelegate.put("消除", "D");
    initonDelegate.put("消掉", "D");      initonDelegate.put("失去", "D");      initonDelegate.put("失散", "D");
    initonDelegate.put("散", "D");      initonDelegate.put("亡", "D");
    // U
    initonDelegate.put("改变", "U"); initonDelegate.put("变通", "U");    initonDelegate.put("修改", "U");
    initonDelegate.put("修整", "U");   initonDelegate.put("变动", "U");   initonDelegate.put("变化", "U");
    initonDelegate.put("整改", "U");   initonDelegate.put("改动", "U");   initonDelegate.put("改", "U");
    initonDelegate.put("变", "U"); initonDelegate.put("更改", "U");  initonDelegate.put("更新", "U");
    initonDelegate.put("出入", "U");  initonDelegate.put("变样", "U");  initonDelegate.put("猫腻", "U");
    // Q 稍后全部文件化去loop
    initonDelegate.put("查看", "Q"); initonDelegate.put("查阅", "Q");  initonDelegate.put("反馈", "Q");
    initonDelegate.put("反应", "Q");   initonDelegate.put("等于", "Q");  initonDelegate.put("提交", "Q");
    initonDelegate.put("公布", "Q");   initonDelegate.put("照应", "Q");   initonDelegate.put("报告", "Q");
    initonDelegate.put("回答", "Q");  initonDelegate.put("回应", "Q");   initonDelegate.put("返回", "Q");
    initonDelegate.put("回执", "Q");  initonDelegate.put("检查", "Q");   initonDelegate.put("检阅", "Q");
    // T
    initonDelegate.put("触发", "T"); initonDelegate.put("反应", "T");  initonDelegate.put("纠缠", "T");
    initonDelegate.put("爆发", "T");initonDelegate.put("制动", "T");
    // X
    initonDelegate.put("探索", "X");    initonDelegate.put("寻找", "X"); initonDelegate.put("索引", "X");
    initonDelegate.put("搜索", "X");  initonDelegate.put("探寻", "X"); initonDelegate.put("查询", "X");
    initonDelegate.put("查找", "X");     initonDelegate.put("搜寻", "X"); initonDelegate.put("发现", "X");
    // H
    initonDelegate.put("主观", "H"); initonDelegate.put("主体", "H"); initonDelegate.put("主导", "H");
    initonDelegate.put("能动", "H");
    // F
    initonDelegate.put("客观", "F");   initonDelegate.put("综合", "F");  initonDelegate.put("全面", "F");
    initonDelegate.put("替代", "F");
}

@SuppressWarnings("unchecked")
        public void init_SMV(App NE) throws NoSuchFieldException
    , InstantiationException, IllegalAccessException, ClassNotFoundException {
    Field[] fields = NE.app_S.getClass().getFields();
    for (int i = 0; i < fields.length; i++) {
        if (null != fields[i]) {
            String string = fields[i].getName();
            S_logger.Log.logger.info("" + string);
            Class<?> type = fields[i].getType();
            String typeNanme = type.getClass().getPackage().getName();
            S_logger.Log.logger.info("" + typeNanme);
            _SMV.put(string, new Object());
            _SMQ.put(string, typeNanme);
        }
    }
```

```
    }
    public Object getObject(String key) {
        if (_SMV.containsKey(key)) {
            return _SMV.get(key);
        }
        return null;
    }
    @SuppressWarnings("unchecked")
            public void putObject(String key, Object object) {
        _SMV.put(key, object);
    }
    //later
}
```

3 时函数应用模块

--主要用在频率滤波上

```
**********************************************************************************************
package test.java.InterfaceTest.timeNorms.jniLYGAFDCDFFT;
import jniLYGAFDCDFFT.LYGAFDCTDFFT_F;
import org.junit.jupiter.api.Test;
import S_logger.Log;
class LYGAFDCTDFFT_FTest {
        public static void main(String[] argv) {
                LYGAFDCTDFFT_FTest lYGAFDCTDFFT_FTest = new LYGAFDCTDFFT_FTest();
                lYGAFDCTDFFT_FTest.all();
        }
        @Test
        void all() {
                S_logger.Log.logger.info("" + "罗瑶光时微分 时叠积 函数 在 主流滤波中 的 观测和优化 内积噪"
                        + " 方式 真实应用方式");
                // 罗瑶光个人著作权文件 时量子解析公式
                // |tero(x)>+|tcol(x)> = (deta[t0-t1]/deta1(t0-t1)) * (m/p)= 1;
                // 时量子解析公式 跟进解析推导 <76页纸 时流噪公式 N = e ** (i * Pi) +-
                // 2cosAcosB;
                // 时量子解析公式 跟进解析推导 <10页纸 时能公式 E = f[d] * f[d] * i * di *
                // deta;
                // 时量子解析公式 跟进解析推导 <3页纸 虚数公式 di= 1/（2 * 根号i） * sin(A - B);
                // 其他这里略,解析推导公式 永不展示，吸取我当年开源德塔分词的教训。单挑70万社会工程人。
                // 公式测试
                // 初始化 函数，非jni的普通java class
                LYGAFDCTDFFT_F lYGAFDCTDFFT_F = new LYGAFDCTDFFT_F();
                // 初始化2维数组 input 为输入数组， out为之前保留的数据。rangeleft
                // 为起始位置，滤数组中的巨大低频噪。
                float[] input = new float[32];
                LYGAFDCTDFFT_F.cp[] out = new LYGAFDCTDFFT_F.cp[32];
                for (int i = 0; i < 32; i++) {
                        input[i] = (float) (Math.random() * 10000);
                }
                // 虚构空间变换 时量子解析公式在 傅立叶计算中的 内积噪表达
                // (2 * out[j].ca * out[j].cb - 1) 观测
```

```
                // 降维状态观测
                float[] fit = lYGAFDCTDFFT_F.jniLYGFIT(input, out, 4);
                // 虚构时能 傅立叶计算中的 内积噪能 (2 * out[j].ca * out[j].cb - 1) 观测
                // 降维状态观测 能量空间分解过程中 产生的 噪能观测。
                float[] ffn = lYGAFDCTDFFT_F.jniLYGFLTn(input, out, 4);
                // 虚度时能 傅立叶计算中的 内积噪能 (2 * out[j].ca * out[j].cb - 1)
                // 进行 l(f(n)) 变换为 l.f.n 加速观测
                // 降维状态观测 极速观测
                float[] flt = lYGAFDCTDFFT_F.jniLYGFLT(input, out, 4);
                // 薛定谔虚度漂移 傅立叶计算中的 内积噪能向量叠积 (2 * out[j].ca * out[j].cb - 1)
                // 降维状态观测
                float[] fxet = lYGAFDCTDFFT_F.jniLYGFxET(input, out, 4);
                // 时函数 傅立叶计算中的 内积噪能总能 (2 * out[j].ca * out[j].cb - 1)
                // 进行 进行 l(f(n)) 变换为 l.f.n 后 复变空间分解，可修正 ffl 复变乘积参数。
                // 增维观测 偏微分细化跟进计算
                float[] fflA = new float[input.length];
                float[] fflBI = new float[input.length];
                float[] ffl = lYGAFDCTDFFT_F.jniLYGFET(input, fflA, fflBI,
                        out, 4);
                // trif later
                // 输出
                S_logger.Log.logger.info("" + "--input");
                for (int i = 0; i < 32; i++) {System.out.print("--" + input[i]);}
                S_logger.Log.logger.info("" + "--");
                S_logger.Log.logger.info("" + "--虚构空间变换");
                for (int i = 0; i < 32; i++) {System.out.print("--" + fit[i]);}
                S_logger.Log.logger.info("" + "--");
                S_logger.Log.logger.info("" + "--虚构时能");
                for (int i = 0; i < 32; i++) {System.out.print("--" + ffn[i]);}
                S_logger.Log.logger.info("" + "--");
                S_logger.Log.logger.info("" + "--虚度时能");
                for (int i = 0; i < 32; i++) {System.out.print("--" + flt[i]);}
                S_logger.Log.logger.info("" + "--");
                S_logger.Log.logger.info("" + "--薛定谔虚度漂移");
                for (int i = 0; i < 32; i++) {System.out.print("--" + fxet[i]);}
                S_logger.Log.logger.info("" + "--");
                S_logger.Log.logger.info("" + "--时函数");
                for (int i = 0; i < 32; i++) {System.out.print("--" + ffl[i]);}
                S_logger.Log.logger.info("" + "--");
                S_logger.Log.logger.info("" + "--时函数 虚度分解A");
                for (int i = 0; i < 32; i++) {System.out.print("--" + fflA[i]);}
                S_logger.Log.logger.info("" + "--");
                S_logger.Log.logger.info("" + "--时函数 虚度分解BI");
                for (int i = 0; i < 32; i++) {System.out.print("--" + fflBI[i]);}
        }
}
*****************************************************************************************
package jniLYGAFDCDFFT;
import P.wave.proportion.Proportion_X_newXY;
```

```
import S_A.pheromone.IMV_SQI;
import java.util.Map;
//这个算法纪念罗瑶光先生逝去的青春。
@SuppressWarnings("unused")
public class LYGAFDCTDFFT_F {
	/*核心蝶形内积公式DIT 见 output 部分*/
	public cp[] log3ffdit(cp[] cps) {
		cp[] output = new cp[cps.length];
		for (int i = 0; i < cps.length; i++) {
			output[i] = new cp();
		}
		int n = cps.length;
		if (n < 2) {
			output = new cp[1];
			output[0] = cps[0];
			return output;
		}
		int part = n >> 1;
		cp[] even = new cp[part];
		cp[] odd = new cp[part];
		cp[] cpi = new cp[part];
		for (int i = 0; i < part; i++) {
			even[i] = cps[i * 2];
			odd[i] = cps[i * 2 + 1];
			cpi[i] = odd[i];
		}
		cp[] a = log3ffdit(even);
		cp[] ab = log3ffdit(odd);
		cp[] abc = log3ffdit(cpi);
		double d_n = cps.length;
		double pi = 3.1415926;
		double o_d_n = 1 / d_n;
		double pi_o_d_n = 2 * pi * o_d_n;
		float c = (float) Math.cos(pi_o_d_n);
		float s = (float) Math.sin(pi_o_d_n);
		float i = s;
		float[] lcc = new float[n];
		float[] lss = new float[n];
		float[] lii = new float[n];
		float cc = c;
		float ss = s;
		float ii = i;
		for (int j = 0; j < part; j++) {
			float kcc = c * cc - s * ss;
			float kss = s * cc + c * ss;
			float kii = i * cc - c * ii;
			cc = kcc;
			ss = kss;
			ii = kii;
```

```
                lcc[j] = cc;
                lss[j] = ss;
                lii[j] = ii;
            }
            for (int j = 0; j < part; j++) {
                c = lcc[j];
                s = lss[j];
                i = lii[j];
                double ccc = a[j].a;
                double sss = a[j].b;
                double iii = a[j].cpi;
                double kss = s * ccc + c * sss;
                double kcc = c * ccc - s * sss;
                double kii = i * ccc - c * iii;
                output[j].a = ab[j].a + kcc;
                output[j].ca = ab[j].ca + ccc;
                output[j].cb = ab[j].cb + c;
                output[j].b = ab[j].b + kss;
                output[j].cpi = ab[j].cpi + kii;
                output[j].sa = ab[j].sa + sss;
                output[j].sb = ab[j].sb + s;
                output[j + part].a = abc[j].a - kcc;
                output[j + part].ca = abc[j].ca - ccc;
                output[j + part].cb = abc[j].cb - c;
                output[j + part].b = abc[j].b - kss;
                output[j + part].cpi = abc[j].cpi - kii;
                output[j + part].sa = abc[j].sa - sss;
                output[j + part].sb = abc[j].sb - s;
            }
            return output;
        }
        public cp[] log3ffdot(cp[] cps) {
            cp[] output = new cp[cps.length];
            for (int i = 0; i < cps.length; i++) {
                output[i] = new cp();
            }
            int n = cps.length;
            if (n < 2) {
                output = new cp[1];
                output[0] = cps[0];
                return output;
            }
            int part = n >> 1;
            cp[] even = new cp[part];
            cp[] odd = new cp[part];
            cp[] cpi = new cp[part];
            for (int i = 0; i < part; i++) {
                even[i] = cps[i * 2];
                odd[i] = cps[i * 2 + 1];
```

```
            cpi[i] = odd[i];
        }
        cp[] a = log3ffdit(even);
        cp[] ab = log3ffdot(odd);
        cp[] abc = log3ffdot(cpi);
        double d_n = cps.length;
        double pi = 3.1415926;
        double o_d_n = 1 / d_n;
        double pi_o_d_n = 2 * pi * o_d_n;
        float c = (float) Math.cos(pi_o_d_n);
        float s = (float) Math.sin(pi_o_d_n);
        float i = s;
        float[] lcc = new float[n];
        float[] lss = new float[n];
        float[] lii = new float[n];
        float cc = c;
        float ss = s;
        float ii = i;
        for (int j = 0; j < part; j++) {
            float kcc = c * cc - s * ss;
            float kss = s * cc + c * ss;
            float kii = i * cc - c * ii;
            cc = kcc;
            ss = kss;
            ii = kii;
            lcc[j] = cc;
            lss[j] = ss;
            lii[j] = ii;
        }
        for (int j = 0; j < part; j++) {
            c = lcc[j];
            s = lss[j];
            i = lii[j];
            double ccc = a[j].a;
            double sss = a[j].b;
            double iii = a[j].cpi;
            double kss = s * ccc + c * sss;
            double kcc = c * ccc - s * sss;
            double kii = i * ccc - c * iii;
            output[j].a = ab[j].a + kcc;
            output[j].ca = ab[j].ca + ccc;
            output[j].cb = ab[j].cb + c;

            output[j].b = ab[j].b + kss;
            output[j].cpi = ab[j].cpi + kii;
            output[j].sa = ab[j].sa + sss;
            output[j].sb = ab[j].sb + s;
            output[j + part].a = abc[j].a - kcc;
            output[j + part].ca = abc[j].ca - ccc;
```

```
                        output[j + part].cb = abc[j].cb - c;
                        output[j + part].b = abc[j].b - kss;
                        output[j + part].cpi = abc[j].cpi - kii;
                        output[j + part].sa = abc[j].sa - sss;
                        output[j + part].sb = abc[j].sb - s;
                }
                return output;
        }
***
        }
        public float[] jniLYGPDN(float[] input, int rangeLeft) {
                int height = 80;
                float[] bili1k = Proportion_X_newXY.newXY(input, 0 + input.length, 80);
                float[] output = new float[161];
                for (int i = rangeLeft; i < bili1k.length; i++) {
                        int temp = (int) bili1k[i];
                        int tempHeight = temp + height;
                        output[tempHeight] += temp;
                }
                return output;
        }
}
```

3 元基变换应用模块 这里略

--主要用在频率滤波上 和 图片色阶分析上（之前著作权作品已经包含，如骨折分析等，此处略）

--主要用在频率滤波（之前著作权作品已经包含，如骨折分析等，此处略）

--我会在操作说明文档中的图解和流程图例逻辑部分进行描述。

个人著作权人, 第一作者: 罗瑶光, 1985年5月25日出生，性别男, 湖南省浏阳市人.
--浏阳德塔软件开发有限公司-创始人，法人，总经理-- 永久非盈利--
yaoguangluo@outlook.com, 313699483@qq.com, 2080315360@qq.com,
（lyg.tin@gmail.com2018年后因G网屏蔽不再使用）
15116110525-（唯一标识 2018年2月开始使用此手机卡至今从未拓机，欠费和关机，手机是实名身份证发票机，警惕我家附近类似GSM电子狗频率充斥欺诈）
430181198505250014, G24402609, EB0581342
204925063, 389418686, F2406501, 0626136
当前居住地址: 湖南省 浏阳市 集里街道 神仙坳社区 大塘冲路一段 208号阳光家园别墅小区 第十栋别墅 第三层
--最近思考家的地址从2009年的北岭美林花苑9号 改成石霜路阳光花园9号，又石霜路阳光花园10号，又大塘路阳光家园10号，再大塘路一段阳光家园第10栋到现在集里大塘冲路一段208阳光家园别墅小区第10栋，我在想，频繁换名，有问题，于是准确介绍下家地址。
--湖南省浏阳市集里街道-大塘冲路一段208阳光家园别墅小区第10栋--，位于大塘冲路一段208阳光家园小区内的别墅房区入口内左侧，靠近美林花苑入口大门（美林花苑小区内见到阳光家园的最前面金属围栏后便是(10号-面朝建筑左户-罗瑶光家)和（11号-面朝建筑右户）2户联体别墅），美林花苑小区位于大塘冲路一段的十字路口，十字路线的以点轴心斜对面是--浏阳市大塘实验(普特融合)小学建筑--。
--身份证上地址2011年后姑姑罗庆华一家入住。中国身份证件标注地址是户籍地址不是居民地址，罗瑶光先生2018年后一直居住在阳光家园家的第三层。